

CLIENT-CENTRIC SERVICE DESK AND REPORT OPTIMISATION: ENHANCING PROJECT TRANSPARENCY IN SOFTWARE CONSULTANCY THROUGH CLOCKIFY INTEGRATION

By

Jordan Eade

(c3350651)

Supervised By

Max Wroe

(CTO of Jaegersoft pty ltd)

Research Report

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Software Engineering (Honours).



School of Information and Physical Sciences
The University of Newcastle
Callaghan, New South Wales 2308, Australia
November 2024

Acknowledgements

I would like to acknowledge first my course coordinator, Dr Kyle Harrison for his support and direction through this honours project. I would also like to thank my industry supervisor, Max Wroe from Jaegersoft for his guidance and assistance with any questions and concerns. Finally, I would like to thank my industry client, Jaegersoft, headed by Vivek Jayachandran and Max Wroe for providing me with the opportunity to work on this project and for their trust and support throughout the entire process.

Contents

List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Background	2
1.1.1 Problem Background	2
1.1.2 Client Background	2
1.2 Problem Statement	2
1.3 Research Objectives	3
1.4 Scope	3
1.4.1 Inclusions	4
1.4.2 Exclusions	4
1.5 Significance	4
1.6 Structure of the Report	4
1.7 Summary	5
2 Literature Review	6
2.1 Background	7
2.2 Review	8
3 Methodology and Design	11
3.1 Project Methodology	12
3.1.1 Software Methodology	12
3.1.2 Design Specifications	12
3.2 Research Methodology	21
3.2.1 Research Design	21
3.2.2 Data Collection	22
3.2.3 Data Analysis	23
3.2.4 Validation and Verification	24
3.3 Summary	24

4	Implementation and Evaluation	26
4.1	Project Implementation	27
4.1.1	Development Process	27
4.1.2	Implementation Details	28
4.1.3	Development Challenges	32
4.2	Project Evaluation	36
4.2.1	Unit & Integration Testing	36
4.2.2	Usability Testing	37
4.2.3	User Acceptance Testing	39
4.2.4	Evaluation of Critical Success Criteria	40
4.2.5	Limitations	41
4.2.6	Future Improvements	41
4.3	Summary	42
5	Results and Discussion	43
5.1	Usability Testing Results	44
5.1.1	Observational Metrics	44
5.1.2	SUS Questionnaire Data	48
5.1.3	Discussion Data	49
5.2	Concurrent Usage Results	50
5.3	Summary	52
6	Conclusions	54
6.1	Summary	55
6.2	Contributions	55
6.3	Implications	56
6.4	Limitations	56
6.5	Recommendations	57
	References	58

List of Figures

3.1	Use Case Diagram	13
3.2	Generate Report Sequence Diagram	14
3.3	Approve Report Sequence Diagram	15
3.4	Architecture Diagram	16
3.5	Component Diagram	18
3.6	Database Diagram	19
4.1	Unit Test Coverage Report	37
5.1	Time Taken to Complete Tasks by Tester	45
5.2	Percentage of Clicks as Errors by Task	46
5.3	Time Taken per Click by Task	47
5.4	Time taken to generate a report	51
5.5	Cost per month with 5 projects	52

List of Tables

5.1	Scenario Testing Results	44
5.2	SUS Questionnaire Results	48
5.3	Sentiment Analysis	49
5.4	Comparison of Manual and Automated Reporting Times	51

Abstract

It is the justified expectation of clients in the software engineering field to be kept up to date on the progress of their projects throughout the development process. This is typically achieved through status reporting, a regular communication used to provide information on the state of the project. The manual production of these reports can be time-consuming and reduce the development capacity of a business. This research investigates the viability of an automated solution for report generation using data from Clockify, a time-tracking application. The research aims to determine whether an automated solution is suitable for status reporting, whether it is more efficient and cost-effective and whether it can meet the needs of the client in this context. The project will involve the development of a proof of concept application which will be used to test the viability of the solution. The research will be conducted alongside a small software consultancy and test the solutions effectiveness using the consultancy's clients with retainer-based contracts. The research used a mixed-methods approach to investigate the research objectives through the use of a usability testing session and an assessment of the application through a period of concurrent use. The developed application achieved a 75% reduction in the time taken to generate a status report and a 72% cost reduction in the production of status reports. The usability testing found users reacted positively to the application and found it easy to use. The opinion of the expert users was that the application would be suitable for use as an automated report generation system with some minor improvements to the user interface. The research found that an automated solution for report generation using Clockify data is suitable for status reports, is more efficient and cost-effective for the business and can meet the needs of the client in the context of status reporting.

1

Introduction

This chapter will introduce the project and provide an overview of the problem and its motivation. The research objectives will be discussed along with the scope and significance of the research. The structure of the report will be outlined to provide an overview of the content of the report.

1.1 Background

1.1.1 Problem Background

During the development of any major work, all stakeholders must be kept informed on the progress of the project. In most industries, this is done through the use of status reports. These reports are used to provide a summary of the work that has been completed, the work that is currently being done and the work that is planned to be completed. They can include information on the status of the project and key indicators of project health. This allows stakeholders to understand the progress and health of the project and to make informed decisions based on these reports. These reports need to be completed regularly and with accurate information to ensure that decisions made are based on the most up-to-date information. For many small businesses, this process can be challenging as it requires a lot of time to complete these reports manually. The process involves extracting data from multiple sources, such as project management tools and time tracking tools, and then compiling this data into a report. As a business grows, the number of projects grows and the number of reports that need to be completed increases. This can lead to a significant amount of time being spent on completing these reports.

Software engineering is a heavily corporate field with most clients being established businesses. Due to this, most clients expect regular and effective communication through established project management practices in their interactions with software businesses. One of the most common communication tools used in software engineering is the status report. The majority of software businesses manage multiple projects at once and need to provide regular status reports to their stakeholders. This process can become time-consuming and reduce the earning capacity of the business as time spent on reporting is time not spent on development. Additionally, many clients are not familiar with the technical aspects of the projects they are funding and require reports that are easily understandable for a non-technical audience. This means that effort needs to be put into ensuring that the reports are clear and concise and provide the necessary information to the client.

1.1.2 Client Background

Jaegersoft is a small software consultancy in the Newcastle and Lake Macquarie area. They have been in business for just over 3 years servicing mainly start-up businesses in Newcastle and the surrounding areas. They are a small team of software engineers who specialise in digital transformation and developing bespoke software solutions to meet the needs of their clients. They have recently experienced a significant increase in interest from larger corporations especially in the heavy-industry and mining sector. This has led to an increase in clients and projects they are managing simultaneously.

1.2 Problem Statement

Jaegersoft is currently experiencing significant growth and an increased intake of clients. The team, consisting of a small group of software engineers, manages multiple clients simultaneously. They use Clockify as a time-tracking solution for each project. However, the manual production of weekly status reports to keep stakeholders informed is a time-consuming process. This manual reporting is causing additional stress on the

business, reducing its development capacity, and limiting its ability to take on new clients. Additionally, many of their clients are not familiar with the technical aspects of their projects and require reports that are easily understandable for a non-technical audience. This necessitates the use of clear and concise metrics to describe the progress and health of the project. It is also crucial that the information in these reports is accurate and up-to-date to ensure informed decision-making based on the most current data.

Challenge 1: Clients need to be provided with regular status reports to keep them informed on the progress of their projects.

Challenge 2: Manually developing these reports is time-consuming and is reducing the development capacity of the business.

Challenge 3: The reports need to be accurate and up-to-date to ensure that decisions are made based on the most current information.

Challenge 4: The reports need to be easy to understand and provide the necessary information to the client.

1.3 Research Objectives

The objective of this research as a whole is to investigate the use of automation in the production of status reports for a small software consultancy. The research will investigate the viability of an automated solution for report generation using data from Clockify. The research will judge the suitability of this solution by testing whether it is more efficient than manual reporting, whether it is more cost-effective for the business and whether it meets the needs of clients.

Obj 1: Is an automated solution for report generation using Clockify data suitable for status reports?

Obj 2: Is an automated solution more efficient for the business?

Obj 3: Is an automated solution more cost effective for the business?

Obj 4: Can an automated solution meet the needs of a client in the context of status reporting?

1.4 Scope

The scope of this research is to investigate the viability of an automated solution for report generation using data from Clockify. The research will be conducted in the context of a small software consultancy and will investigate the efficiency of this solution on clients with retainer-based contracts. As part of this research, a proof of concept application will be developed to test the viability of the solution in the given context. The research will only investigate the use of automation in the software engineering field and will not involve the use of any other time-tracking or project-management tools.

1.4.1 Inclusions

- Investigate the viability of an automated solution for report generation using data from Clockify, a time-tracking application.
- Investigate the efficiency of an automated solution for a small software consultancy.
- Investigate the effectiveness of the solution in retainer-based contracts.
- Develop a proof of concept application to test the viability of the solution.

1.4.2 Exclusions

- Research will not include the use of any other time tracking or project management tools.
- Research will not investigate the use of automation in other industries.
- Research will not investigate the use of automation in other types of contracts such as fixed-price contracts.

1.5 Significance

This research aims to investigate the viability of an automated solution for report generation using data from Clockify. The results of this research will be used to improve Jaegersoft's business processes with a more efficient and cost-effective approach to status reporting. The findings of this research will be used to inform the viability of a production-level application to replace the manual reporting process for Jaegersoft. The research will be significant to the wider body of software engineering as it will provide insights into the use of automation in the context of external status reporting. The research will judge the effectiveness of status reporting using data from a time-tracking tool and provide a case study for other small software consultancies investigating methods to automate their process. The findings of this research will also give insights into the priorities of clients in the context of status reporting as part of testing the effectiveness of the solution in meeting client needs.

1.6 Structure of the Report

This report will explore the use of automation in the production of status reports for a small software consultancy. The report will be structured as follows:

Chapter 1: Introduction — This chapter will introduce the project and provide an overview of the research objectives.

Chapter 2: Literature Review — This chapter will review the current literature on status reporting and automation in status reporting.

Chapter 3: Methodology and Design — This chapter will outline the methodology used in the research and the design of the software.

Chapter 4: Implementation and Evaluation — This chapter will detail the implementation and evaluation of the software.

Chapter 5: Results and Discussion — This chapter will present and analyse the results of the research.

Chapter 6: Conclusions — This chapter will summarise the findings of the research and provide recommendations for future research.

1.7 Summary

This project will investigate the viability of an automated solution for report generation using data from Clockify. The research will aim to determine whether an automated solution is suitable for status reporting, whether it is more efficient and cost-effective and whether it can meet the needs of the client in this context. The research will not investigate the use of automation in other industries or other types of contracts such as fixed-price contracts. The background and state of the research field will be explored further in the literature review.

2

Literature Review

This chapter will review the current state of the literature in the area of status reporting. The literature review will focus on the key components of a comprehensive project status update, the common issues with status reporting, the metrics commonly used to provide an objective representation of project health and the effectiveness of visualisation of data in a dashboard format. The review will be used to inform the research objectives and the development of the proof of concept application.

2.1 Background

A literature review was conducted during this project to assess the current state of the literature in this area and identify research gaps to address in this project. It was found that the literature in this area is outdated and not many papers exist on the subject of automated status reporting. This may be due to the topic being more heavily investigated internally by businesses either to keep a competitive advantage or as it addresses a specific situation. The review found that the most recent literature focused on the common problems and effectiveness of status reporting in the industry and manually operated dashboard visualisations. As such there is a gap in the literature for the effectiveness of automation in status reporting. Due to the lack of literature in this area, the review focused on four main questions and their implication on the research being conducted. The questions were:

Question 1: What are the key components of a comprehensive project status update?

Question 2: What are the common issues with status reporting?

Question 3: What metrics are commonly used to provide an objective representation of project health?

Question 4: Is visualisation of data in a dashboard format an effective way to communicate project status?

2.2 Review

In the realm of project management, effective communication of project status is crucial for stakeholder decision-making. Although the status of the research field is quite out of date when discussing the structure and inclusions of an effective status report, there are still valuable insights to be taken. This literature review explores the key components of a comprehensive project status update and the common issues with status reporting. The review also investigates potential automated metrics to provide more objective and accurate representations of project health and a dashboard visualisation format as a suitable presentation of the metrics.

Project status reports are an essential communication tool used by project managers to communicate the current status of a project to stakeholders. In [1], the structure and inclusions of a comprehensive project status report. The textbook describes a status report as a regular communication from the project manager to external stakeholders showing the progress and health of a project as well as early warning for potential problems. The status reports should provide a history of the project for new stakeholders.

Stakeholders use the report to decide on future actions with the project, such as whether preventative action is needed for an unhealthy project or further investment is needed for a project running behind. The source goes on to propose topics that should be included and discussed in a status report including activities and accomplishments, problems encountered, problems solved, problems outstanding, time and expenses, progress and trend analysis and plans for the next period. Each of these topics is to be manually discussed and presented to stakeholders for consideration in a high-quality status report. In [2], different styles of status reporting using a dashboard tool are presented. The dashboard is used to provide the real-time status of the project through the display of metrics and comprehensible graphs. The source explains that clients and stakeholders may have different needs for metrics to be included in the dashboard and that details should be flexible. Although both texts present different mediums for communication, both agree on the purpose of the status report, which is to inform stakeholders about the project's health and provide information to make financial decisions. The second text [2] proposes that the information included in the report should be flexible between different stakeholder groups. However, the purpose of the status report suggests that there should at least be the information proposed by [1] included in the report. This information can be used to inform the necessary components within the proof of concept application to be implemented in the research. The application should provide information on the activities and accomplishments, problems encountered, problems solved, problems outstanding, time and expenses, progress and trend analysis and plans for the next period.

Project status reports written by project managers and staff have the potential to be inaccurate or misleading for many reasons. The most popular opinion within literature is that the most common and impactful cause of inaccuracy is bias. In [3], research is conducted on the impact of statistical errors and reporting bias on the ability of stakeholders to make decisions on the continuation of a project. The paper discusses this specifically in the case of a status report using the traffic light method. Essentially this method reports the status of the project as 'Green' is good, 'Yellow' is okay, and 'Red' is unhealthy. The report found that through both statistical errors and optimistic bias applied by project managers, the probability of correctly perceiving the status of a high-risk project as 'Red' is 48%. The study also states that the higher the risk associated with the

project, the more likely the status is to be reported as healthier than its actual state. This result is supported by the study [4]. This paper explored the possible motivations as well as the frequency and strength of both optimistic and pessimistic bias on the accuracy of project status reporting. This study shows that status reports are most likely to be biased. It is twice as likely that reports are optimistically biased and even with low levels of bias, the combination of this with errors in perception makes reports overly optimistic. It was also found that when bias is applied, only 10–15% of reports in high-risk projects are accurate and only when pessimistic bias is applied. These papers were published 5 years apart but both found similar results. That bias is present in a large majority of status reports and the presence of this bias greatly affected the accuracy of the reports. The data is quite old, being from 2002 and 2007, however, the results are still applicable as the motivations for this bias, which [4] cites as fear of reprimand and need to impress management, are still present in the field today. It could be argued that the accuracy of the data for both of these studies may not be completely reliable. Both studies are based on data captured through surveys of active project managers, these could easily be affected by bias themselves and are inherently opinionated rather than objective measurements. As such it can be deduced that it is more than likely that project status reports are affected by bias, which has had a great impact on the accuracy of reports and through this the success of project completion. However, the collected data should not be used to surmise exact measurements of the prevalence of bias in status reports. This information is important to the research as it highlights the need for accurate and data-based status reports. It guides the implementation of the proof of concept application by suggesting that the application should be based majorly on data and not on human opinion. This will reduce the risk of human bias in the reports and provide a more accurate representation of the project's health.

One method for reducing bias is to remove or reduce the dependence on human perspective in deciding the status of a project. To do this a set of comprehensive metrics can be used to determine the health of a project. The study [5], created a ranking of the most common metrics discussed in reported literature for Agile and Lean methodology projects. They found that implementation, testing and the development cycle as a whole are most commonly captured in the measured metrics. The 10 most common metrics used in this study were velocity, effort estimate, customer satisfaction, defect count, technical debt, build status, progress as working code, lead time, story flow percentage and velocity of elaborating features. The study [6], surveyed 24 development companies to identify a list of metrics useful in agile methodology projects. They found their top 10 metrics as delivery on time, work capacity, unit test coverage, percentage of adopted work, bug correction time from new-to-closed, sprint level effort burn down, velocity, percentage of found work, open defect severity index and focus factor. Finally, the study [7] applied the Goal-Question-Metric approach to develop their own set of 11 metrics based on the goals of project owners. The metrics, mostly based on product backlog items which they refer to as PBIs are as follows, added PBIs, changed PBIs, enhancement rate, estimation shift, expenses prognosis, effort at risk, priority shift, project size, project size remaining, rejected PBIs, scope prognosis, software quality and time prognosis. Each of these studies presents a list of metrics useful for agile methodology projects, the common themes throughout their metrics are measuring performance and efficiency, quality and defects, progress and predictability and risk and change management. Through these studies, a list of metrics can be extracted that commonly occur in the top 10 metrics in each study or are closely related. These include velocity, effort estimate, defect count, unit test coverage, lead time and added PBIs. These metrics will be

useful in the development of the proof of concept application as they provide insight into the information that is valued in status reports. Of the extracted metrics, most are related to the time taken to complete work or the expected time to complete work. This suggests that these elements are important to stakeholders in the context of status reporting. The application should focus on metrics that relate to these areas.

The presentation of information in a status report is an important factor in the ability of users to understand the information in front of them. The study [8] proposed a tool for use in the construction industry that provides visualisations of important metrics depicting cost and progress estimation. The study evaluated the effectiveness of this tool through its use in a real construction project through the design and construction phases to track progress. The research presents qualitative assessments of the system's improvements in the construction process which are hard to compare to other data or verify conclusions with. However, the research indicates that the system was effective in communicating important metrics depicting project progress and cost through visualisations similar to a dashboard. This conclusion is backed in [9] as they tested the effectiveness of a dashboard tool DashVis. The tool is used to provide dashboard visualisations of progress in an agile software project internally to a team of engineers. The research used a team of software students over 2 weeks working on a project to judge the effectiveness of the tool in keeping the team up to date on the progress of the project. Results showed that the team found the dashboard visualisation useful for tracking progress, in particular the provided burn-down chart. Both of these studies propose dashboard visualisation tools for tracking project progress. It is important to notice that [8] is in the context of a construction project so the actual visualisations and metrics would not be useful in the software engineering field. However, the premise of depicting progress using a visual tool is still applicable. Both studies indicate that the use of dashboard visualisation tools is beneficial for depicting project progress and [8] shows that this method of tracking is capable of assisting in informed project management decisions. From the findings of these studies, a dashboard visualisation would be an effective method of presenting the metrics to be included in status reporting for the research to be conducted.

In conclusion, the literature review emphasises the critical importance of project status reporting in project management. Despite the outdated nature of some of the foundational research, the key insights remain relevant, particularly concerning the structure and inclusions of status reports and the challenges posed by reporting bias. Status reports are essential to keep project stakeholders informed on the health of a project but have previously been drastically affected by reporting bias. Automated metrics are a suitable solution with many measurable and verifiable metrics available to depict project health. The presentation of these metrics in a dashboard format is an effective format for communicating project progress and health to stakeholders. The findings of the literature review will be used to inform the development of the proof of concept application to be implemented in the research. The application will focus on providing a comprehensive set of metrics to depict project health and progress and present these metrics in a dashboard format to stakeholders. The application will aim to reduce the risk of bias in status reporting and provide a more accurate representation of the project's health.

3

Methodology and Design

This chapter will outline the design of the proof of concept application and the methodology used in the research. The design specifications of the application will be discussed along with the software methodology used in the development of the application. The research methodology will be discussed including details on the data collection and analysis methods used in the research.

3.1 Project Methodology

This section will present the implementation details of the proof of concept application.

3.1.1 Software Methodology

The software methodology used in this project is an adapted version of the Agile methodology. The Agile methodology is a software development methodology that focuses on iterative development and incremental delivery. It is based on the Agile Manifesto which outlines the core values and principles of Agile development. The Agile methodology is characterised by its focus on collaboration, flexibility and adaptability. It is designed to be responsive to change and to deliver value to the customer quickly and efficiently. This is well suited to this project as it allows for the development of the proof of concept application iteratively and incrementally. This will allow for the application to be developed quickly and efficiently and for changes to be made to the application as needed. The Agile methodology will also allow for the application to be developed in close collaboration with Jaegersoft to ensure that the application meets their needs and requirements. The Agile methodology was adapted for this project to perform iterations on milestones of the project. Each sprint lasted 2 weeks and was used to develop a specific feature of the application. This suited the needs of the research team to be flexible with workload between sprints and allowed for constant feedback from the client. A sprint retrospective was used at the end of each sprint to review the work completed and to identify areas for improvement in the next sprint. This worked well to keep the client involved in the development process and to ensure that the application met their needs.

3.1.2 Design Specifications

The design specifications section will provide details on the design of the application. The use cases addressed by the application will be discussed with sequence diagrams explaining complex and important use cases in more detail. The architecture of the application will be discussed with a diagram showing the components of the application and how they interact and are deployed. The database design will also be discussed showing the data model the application uses to store data, and finally, the components of the application and their interaction will be presented with a UML component diagram.

Use Cases

The application is intended to automate the production, maintenance and communication of status reports for a small software consultancy. As part of these goals, the application interacts with three types of users with unique needs. Firstly, the ‘Client’ user. The client user represents the party funding the project and is the primary recipient of the status reports. They wish to interact with the application to receive information on the progress of their project. Secondly, the ‘Project Manager’ user. The project manager is an employee of Jaegersoft and is responsible for managing the project and ensuring that the project is delivered on time and within budget. They are in charge of the accuracy and successful delivery of the reporting to the client. Finally, the ‘Super Admin’ user. The super admin user is an employee of Jaegersoft who is responsible for the management of the application and the resources inside the application. They are responsible for ensuring

all users have the correct permissions and access within the application. Figure 3.1 shows the UML use case diagram depicting the different user types and their interactions with the application.

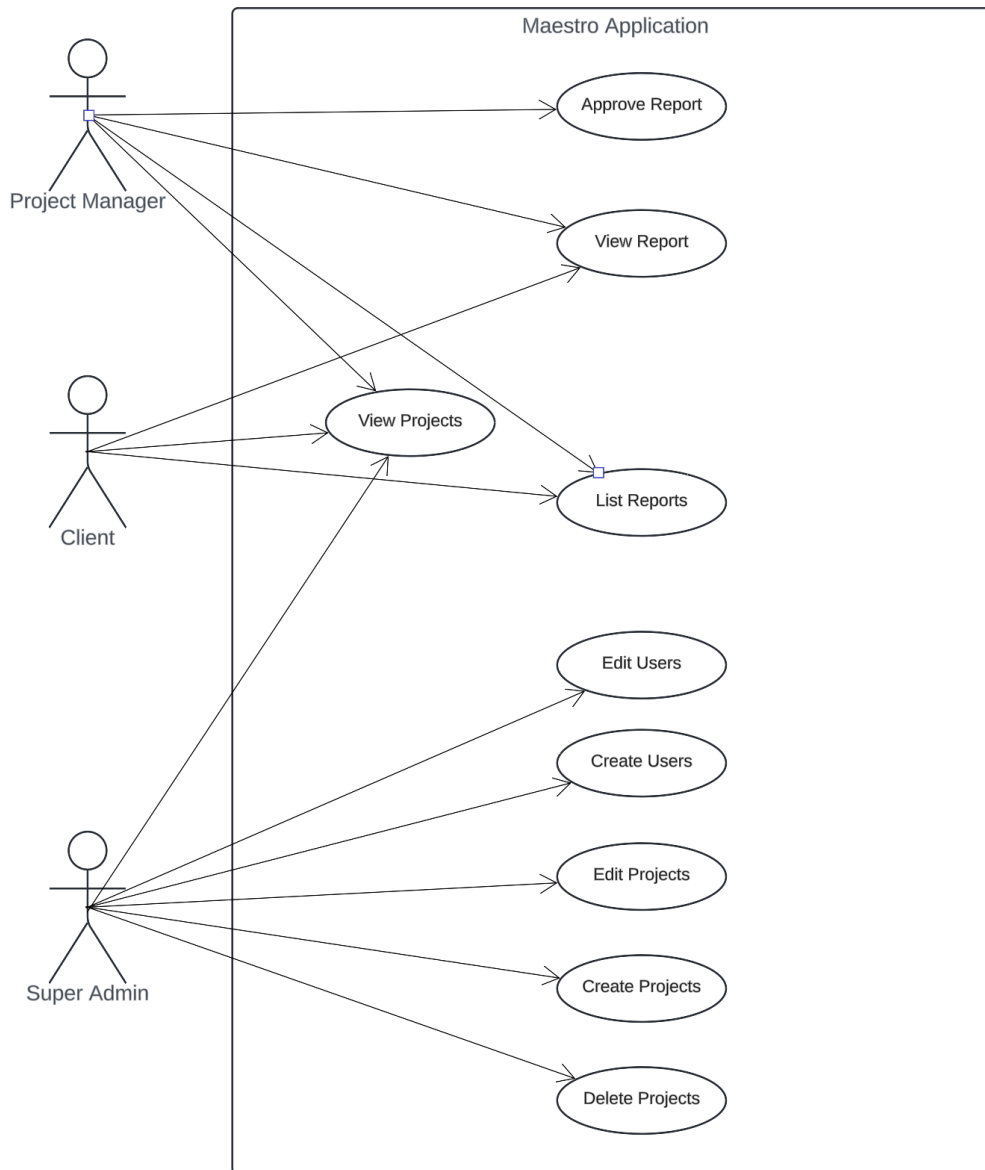


FIGURE 3.1: Use Case Diagram

One major use case of the system is to automatically generate status reports, this use case does not occur in Figure 3.1 as it is not specific to a user. This is a scheduled action that occurs regularly each week using machine-to-machine interaction.

Sequence Diagrams

The scheduled process to generate progress reports in the application is complex. The process involves the use of multiple technologies in coordination to produce the result. The process is initiated by a scheduled AWS Eventbridge event at 11:59 pm each Wednesday. This event is used to trigger a lambda function that will call the application API to perform the report generation. The application will then communicate with the Clockify API to retrieve the necessary data for the report and store the report in the database. This process is outlined in further detail in Figure 3.2.

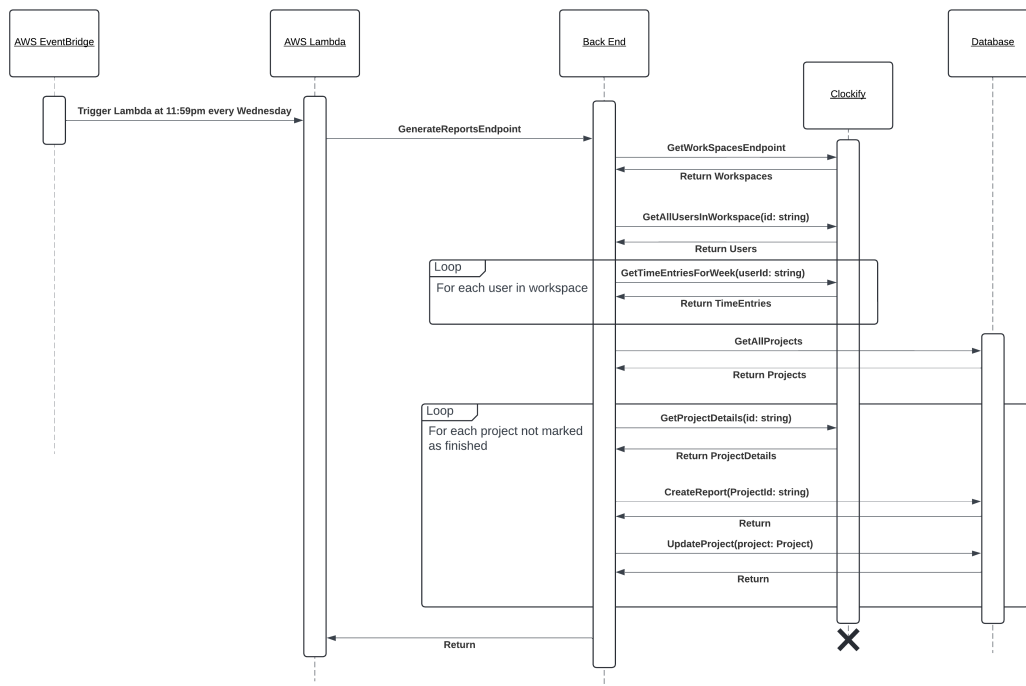


FIGURE 3.2: Generate Report Sequence Diagram

The report approval process is a key feature of the application. This process allows the project manager to review the report generated by the system and approve it for sending to the client. The process is initiated by the project manager logging into the application and viewing the generated report. The project manager can then approve the report for sending to the client. This process is outlined in further detail in Figure 3.3. The process is fairly simple and most other use cases follow a similar pattern of user interaction.

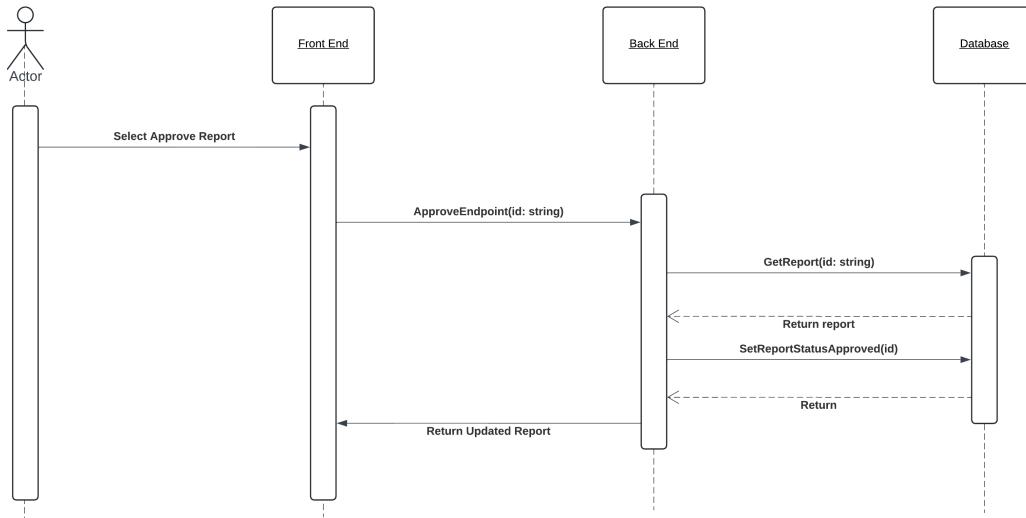


FIGURE 3.3: Approve Report Sequence Diagram

Architecture

The architecture used for this application is a component-based architecture called "Back-end For Front-end". This architectural type allows for the separation of concerns between the front-end and back-end of the application and the ability to select more specialised technologies for each component. The architecture is designed to be scalable and flexible to allow for future extension. The architecture is shown in Figure 3.4.

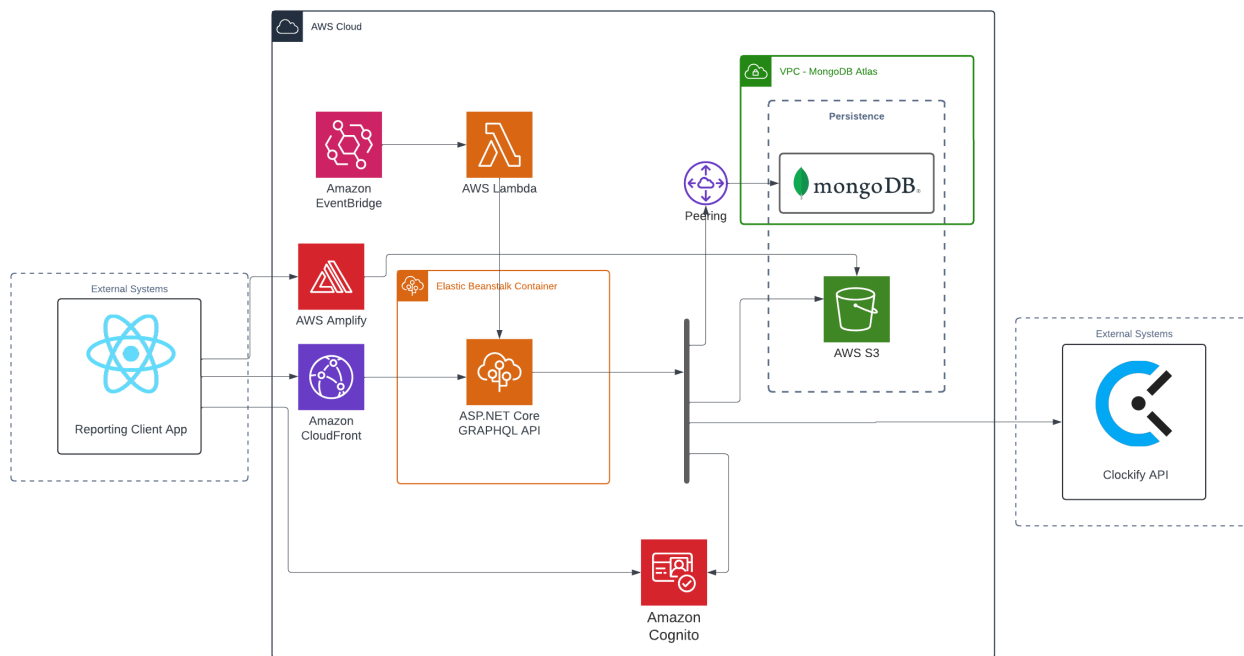


FIGURE 3.4: Architecture Diagram

For the front-end of the application, a Typescript React Single Page Application (SPA) was used. This is a popular framework for use in a front-end application as it allows for the creation of a dynamic and responsive user interface. Vite was used as the build tool for the application as it is a fast and efficient solution for building react applications. Apollo Client was used to handle the communication between the front-end and back-end of the application. This is a popular library that provides many tools to assist with interacting with a GraphQL API including the automated generation of hooks and types as well as state management and an effective cache. The front-end was deployed using AWS Amplify which is an effective service for managing the deployment of front-end applications. For this deployment, it essentially abstracts the storage of the react SPA in an AWS S3 bucket and the retrieval of that SPA as it is served to the browser.

The back-end component of the application was developed using ASP.NET Core. This is a popular C# framework developed by Microsoft. It handles all business logic within the application and integrates with other technologies to perform the critical functionalities of the system. The back-end exposes a GraphQL API using Hot Chocolate, a popular GraphQL framework used in .NET applications provided by ChilliCream. This package allows for the easy creation of a GraphQL API in a .NET application and access to the advanced

capabilities of the technology such as resolvers and subscriptions. The back-end is deployed using AWS Elastic Beanstalk which is a service used for the managed deployment of web applications of any kind. It abstracts the deployment of the application to a server and the management of that server. In this case, the application is deployed to an EC2 instance running the Linux operating system. Elastic Beanstalk handles the deployment of the application to the EC2 instance and the management of the instance itself including scaling and monitoring using a suite of AWS services. An AWS Cloudfront distribution sits in front of the Elastic Beanstalk environment to provide a secure and performant connection to the application. This distribution exposes the endpoints of the application to the internet and provides a secure connection to the application.

The database used for the application was a NoSQL document store database called MongoDB. This technology was chosen as it allows for a flexible data model suitable for future extension and is a performant and reliable database system. MongoDB has its own hosting solution called MongoDB Atlas which was used to host the database. MongoDB Atlas is a fully managed cloud database service that handles the complexity of deploying and managing the database. It provides a reliable and secure database solution that is scalable and flexible to meet the needs of the application.

An AWS Lambda function is used to handle the scheduled report generation process. This function is triggered by an AWS Eventbridge event and calls the API to generate the report regularly. AWS Cognito is used to manage Authentication in the application. All sensitive user information including usernames and passwords are stored in the Cognito User Pool and all authentication requests are handled by Cognito. This abstracts the authentication process from the application and removes the need to store complex encrypted data in the application itself.

Software Components

Inside the back-end application of the system, multiple components interact with each other to provide the necessary functionality. The components are separated by their function and interact with each other to provide the necessary functionality. The components of the application are shown in Figure 3.5.

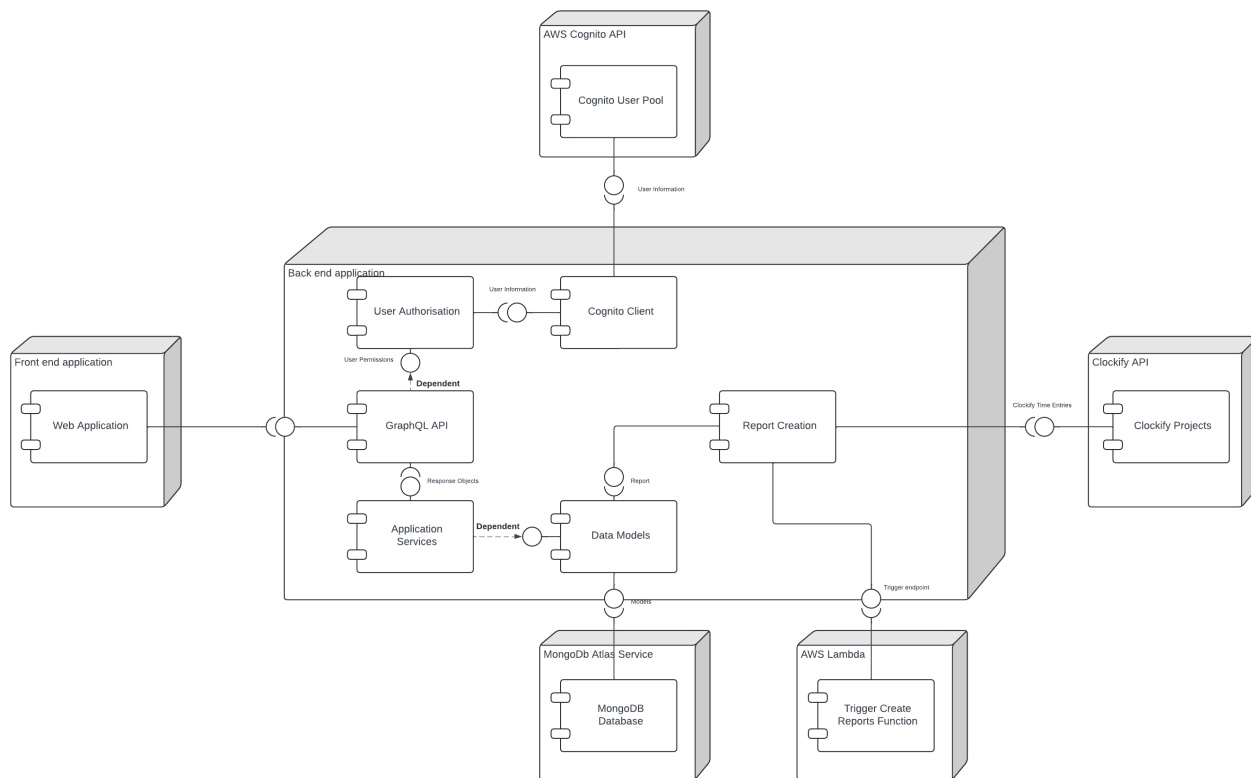


FIGURE 3.5: Component Diagram

The major components of the application are the GraphQL API, the Application Services and the Report Creation components. The GraphQL API exposes the endpoints for the front-end to interact with. Through these endpoints, the User Authorisation component can authenticate and authorise users before the request is passed to the Application Services. The GraphQL API is composed of Query and Mutation classes that define the functionality to be performed for each request. The Application Services component is responsible for handling the business logic of the application. This component exposes a set of services that are available for the GraphQL API component to call. The Application Services component is composed of 2 layers, the Service layer and the Repository layer. The Service layer is responsible for handling the business logic of the application and the Repository layer is responsible for handling the data access logic of the application. Abstracting the data access logic from the business logic allows for better testability in the application as the business logic can be isolated much easier. The Report Creation component is responsible for generating the progress reports using information from the Clockify API. The component is triggered by the external, scheduled AWS Lambda

function and is responsible for retrieving the necessary data from the Clockify API and storing the report in the database.

Database Design

The database used for the application is the NoSQL document store database MongoDB. The use of this technology allows for a flexible data model that can easily be extended and has the capacity to store all types of data required by the application. The database design is shown in an Entity Relationship diagram in Figure 3.6.

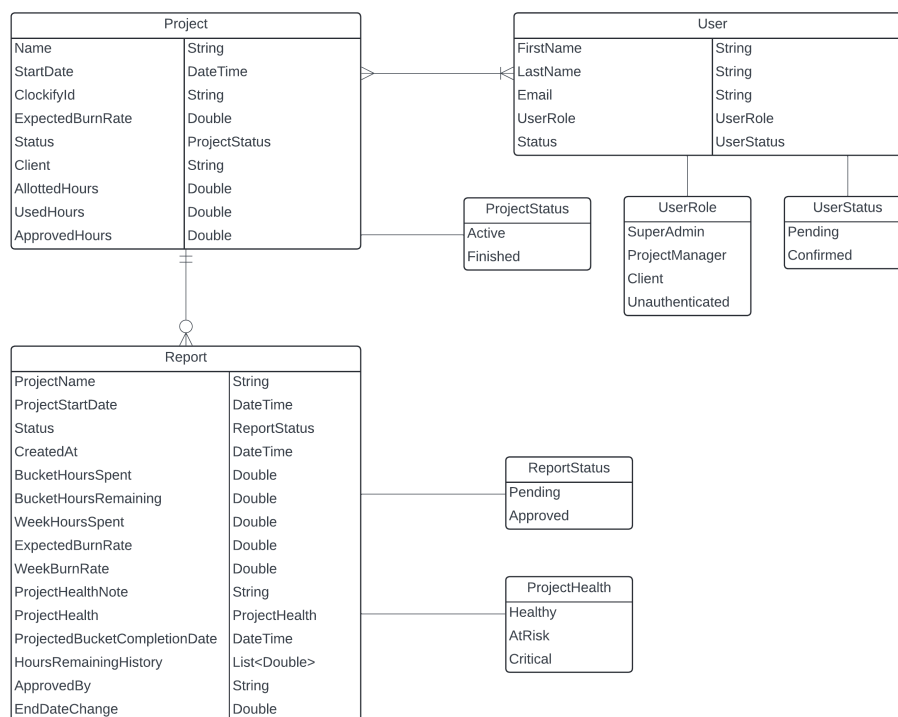


FIGURE 3.6: Database Diagram

The application uses a small and simple schema to store the necessary data. The schema is composed of three collections, the User collection, the Project collection and the Report collection. The User collection stores necessary information about the users of the application including their name, email, role and status where role and status are represented by enumerations. The Project collection stores information about the projects in the application and includes details about the project for display and connection to the correct data source in Clockify. The Report collection stores the generated reports for each project and includes the necessary information to display the report to the client. The schema is simple and effective for the needs of the application and can be easily extended to include more complex data structures if needed. One main advantage of the document store technology and this schema is the efficiency when querying the data. All data for a report is stored in one collection and can be retrieved in one query. This is a major advantage over a relational database where the data would be stored in multiple tables and require more complex, joining queries to retrieve all the necessary data. The document store can still provide some of the benefits of a relational

database including storing relationships between collections. However, it cannot provide the same protections against data duplication and inconsistency that a relational database can provide. This is not a major concern for the application as the data stored is simple and does not require complex relationships between collections.

3.2 Research Methodology

This section outlines the methodologies employed to address the research objectives. It covers the research design, data collection and analysis techniques, as well as the validation and verification methods used to ensure the reliability and accuracy of the findings. The goal of the research in this project was to investigate the viability of an automated solution for report generation using data from Clockify. To do this, the research would need to meet the following objectives:

Obj 1: Investigate the suitability of an automated solution for report generation using Clockify data.

Obj 2: Investigate the efficiency of an automated solution for a small software consultancy.

Obj 3: Investigate the cost effectiveness of the solution in a small software consultancy.

Obj 4: Investigate the ability to meet the needs of a client in the context of progress reporting.

3.2.1 Research Design

The research conducted in the project used a mixed methods approach. This approach was chosen as it allows for the collection of both qualitative and quantitative data to provide a comprehensive answer to the research objectives. The qualitative data collected was used to answer objectives 1 and 4 and the quantitative data to answer objectives 2 and 3.

The qualitative research focused on the usability of the application and the suitability of the application for client-facing status reporting. This research was conducted through a usability testing session where participants were monitored as they completed tasks designed to assess the usability and suitability of the application. The qualitative research was mostly centred on the opinions of the participants and their experience with the application.

The quantitative research focussed on the efficiency and cost-effectiveness of the application. This research was conducted through a period of concurrent use with the manual reporting process by Jaegersoft. The assessment was conducted by Jaegersoft employees who were actively managing projects with different clients with the direction of the research team on what data needed to be collected.

Usability Testing Session Design

The usability testing session was used to assess the usability and suitability of the application for client-facing status reporting. The session was conducted with 5 participants who were employees of Jaegersoft and had knowledge of the manual reporting process used by the company. The participants were active software engineers with 2 currently managing projects with different clients. The participants were chosen as they were representative of the users of the application and could provide valuable feedback on the application's usability and suitability for client-facing status reporting. The session was conducted in a controlled environment with the participants being monitored as they completed tasks designed to assess the usability of the application. The tasks were designed to simulate the process of generating a status report using the application and to assess

the ease of use of the application. After each task, the participants were able to express any thoughts they had on the suitability or usability of the application. Once tasks were completed, the participants were given a System Usability Scale (SUS) survey to assess the usability of the application. Finally, a set of discussion-based questions were used to gather more qualitative data on the participant's experience with the application and their thoughts on the application's suitability for client-facing status reporting.

Concurrent Use Assessment Design

The concurrent use assessment was used to assess the efficiency and cost-effectiveness of the application. The assessment was conducted through a period of concurrent use with the manual reporting process by Jaegersoft. The assessment was conducted by 2 Jaegersoft employees who were actively managing projects with different clients. The employees were given the task of completing a status report using the manual reporting process and the automated solution. The research team directed the employees on the data to be collected and the process to be followed. One employee was directed to generate the automated report using the application before generating the manual report and the other was directed to generate the manual report before generating the automated report. This reduced the possibility of results being skewed based on the order of the reports being generated.

3.2.2 Data Collection

The data collected was a mixture of qualitative and quantitative data. The data collected was used to assess the usability, efficiency, cost-effectiveness and suitability of the application for status reporting. Qualitative data was collected through the usability testing session mostly with questionnaires and organic discussion. Quantitative data was collected through the concurrent use assessment using time and cost metrics as well as the usability testing session using the SUS survey and observational metrics.

Usability Testing Data Collection

During the usability testing session, data was collected on the usability and suitability of the application for status reporting. There were three main methods of collecting data used in the session.

Observational data collection: Observational data was collected by monitoring the participants as they completed tasks designed to assess the usability of the application. The data collected included the time taken to complete each task, the number of clicks made and any errors made by the participants.

Questionnaire data collection: Questionnaire data was collected through the use of a SUS survey. The survey was used to assess the usability of the application and to gather feedback on the application's suitability for status reporting.

Discussion data collection: The opinions and perspectives of the participants were collected through a set of discussion-based questions. These were used to gather more qualitative data on the participant's experience with the application and their thoughts on the application's suitability for progress reporting.

Concurrent Use Assessment Data Collection

During the concurrent use assessment, data was collected on the efficiency and cost-effectiveness of the application. Two Jaegersoft employees were asked to use the automated solution and manual reporting process to generate a status report for projects they were managing. The research team asked them to record the time taken to generate the manual report and the time taken to generate the automated report. The research team also monitored the costs of maintaining the deployed application using the AWS cost explorer tool. The monthly cost of the application was recorded and included in the assessment.

3.2.3 Data Analysis

The collected data fits into 4 categories, observational metrics, SUS questionnaire data, opinionated data from discussions and time and cost metrics.

Observational Metrics

The observational metrics collected during usability testing are quantitative data. The metrics include the time taken to complete each task, the number of clicks made and any errors made by the participants. This data was mainly useful for assessing the usability of the application and the ability of the application to meet the needs of the client in this way. This raw data was analysed to draw more general conclusions about the usability of the application.

Error – Click Ratio: The error-to-click ratio was calculated by dividing the number of errors made by the number of clicks made by the participant for each task. This ratio was used as a measure of complexity in the application.

Time per click: The time taken to complete each task was divided by the number of clicks made by the participant. This was used to find areas of the application that were more complex and required participants to spend more time understanding the information presented to them.

SUS Questionnaire Data

The SUS questionnaire data collected during usability testing is quantitative. The data collected was used to assess the usability of the application and to gather feedback on the application's suitability for status reporting. The average SUS score was calculated and used to assess the overall usability of the application. The answers to each specific question we also used as feedback on specific areas of the application.

Discussion Data

The discussion data collected during usability testing is qualitative. It consists of the opinion and perspective of the participant on the application. The data can be used as feedback on the application's suitability for status reporting and the participant's experience with the application. Sentiment analysis was used to analyse the general feeling of the participants towards the application and thematic analysis was used to identify common themes in the feedback provided by the participants.

Time and Cost Metrics

The time and cost metrics collected during the concurrent use assessment are quantitative data. The data was used to compare the time and cost efficiency of the automated solution and manual reporting process. The time taken to generate the manual report and the automated report was compared to assess the efficiency of the application. From the monthly cost of the application deployment, the weekly cost was calculated. This was included in the cost for the automated reporting process alongside personnel costs before it was compared to the cost of the manual reporting process.

3.2.4 Validation and Verification

To ensure the validity and reliability of the research, several methods were employed:

Pilot Testing

Before conducting the main usability testing session, a pilot test was conducted with a non-testing participant. This helped identify any issues with the testing process, the clarity of the tasks, and the functionality of the application. Adjustments were made based on the feedback from the pilot tester to improve the reliability of the main testing session.

standardised Procedures

A standardised procedure was followed for the usability testing session. This included a consistent set of tasks, instructions, and questions for all participants. By maintaining consistency, the potential for variability in the data collection process was minimized, enhancing the reliability of the results.

Triangulation

Multiple data sources and methods were used to gather information. This included observational metrics, SUS questionnaires, discussion data, and time and cost metrics. Triangulating data from different sources helped validate the findings and provided a more comprehensive understanding of the research objectives.

Concurrent usage

During the concurrent usage assessment, the 2 Jaegersoft employees were directed to use the automated solution and manual reporting process to generate a status report for projects they were managing. One employee was directed to generate the automated report before the manual report and the other was directed to generate the manual report before the automated report. This reduced the possibility of results being skewed based on the order of the reports being generated.

3.3 Summary

The methodology and design of the research and the proof of concept application were outlined in this chapter. The software methodology used in the development of the proof of concept application was an adapted version

of the Agile methodology. The application was designed as a component-based architecture called "Back-end For Front-end" using React Typescript and ASP.NET Core. This architecture allows the application to be scalable and flexible to allow for future extensions. The architecture, database design and software components of the application were discussed in detail.

The research used a mixed methods approach to address the research objectives. The usability testing session was used to assess the usability and suitability of the application for client-facing status reporting. The concurrent use assessment was used to assess the efficiency and cost-effectiveness of the application. The data collected was a mixture of qualitative and quantitative data providing different perspectives to make comprehensive conclusions. The data was analysed using a variety of methods including observational metrics, SUS questionnaire data, opinionated data from discussions and time and cost metrics. To ensure the validity and reliability of the research, several methods were employed including pilot testing, standardised procedures, triangulation and the design of the concurrent usage assessment. The research methodology was designed to provide a complete answer to the research objectives and to ensure the reliability and accuracy of the findings.

4

Implementation and Evaluation

This chapter will detail the implementation of the proof of concept application and the evaluation of the research. The implementation section will detail the development process of the application and the implementation details of the key modules of the application. The evaluation section will detail the method used to evaluate the application and in cases where the data is not part of the research will present the results of the evaluation.

4.1 Project Implementation

The project implementation section will detail the implementation process of the proof of concept application. This will include information on the development process, implementation details and any challenges faced during the implementation of the project.

4.1.1 Development Process

The implementation of the project's infrastructure and 4 key modules consisted of 5 main phases each holding a different responsibility in the development of the application. The phases were as follows:

Foundational Phase: The foundational phase involved the implementation of the basic infrastructure of the application. This included the creation of the front-end and back-end applications and the connection to the MongoDB database. The front-end was developed using Typescript React and the back-end was developed using ASP.NET Core. The connection to the MongoDB database was established using the MongoDB C# driver.

User Interface Phase: The user interface phase involves the initial implementation of the user interface of the application. The foundational components were developed in this phase to be used throughout the rest of the application. The main pages were created with mock data returned from the API and displayed to the user.

Authentication Phase: The authentication phase involved the implementation of the user authentication and authorisation in the application. An AWS Cognito User Pool was created and linked to the application to handle authentication and the login components were developed here. The endpoints of the back-end application was then secured.

External Connection Phase: The external connection phase involved the connection of the application to the Clockify API to replace the mock data with real data.

Automated Report Generation: The automated report generation phase involved the implementation of the scheduled report generation process. An AWS Lambda function was created to trigger the report generation process and the Clockify API was used to retrieve the necessary data for the report. The report was then stored in the MongoDB database.

Each phase was initially allotted 2 sprints to complete. This was adjusted as needed to ensure the application was completed on time and to a reasonable standard. At the end of each sprint, a sprint retrospective was conducted to review the work completed and to identify areas for improvement in the next sprint. Git was used as the version control system with the feature branch workflow to ensure the main branch never contained broken code. Github was used to store and manage the project repository. Using the feature branch workflow, once a feature was complete and ready to be merged with the main branch, a pull request was created and reviewed as part of a formal code review process. This was used to ensure the quality of the codebase and to identify any potential issues before they were merged with the main branch.

Through the development of the application, multiple tools, technologies and practices were used to create a better developer experience and improve productivity. These tools could be easily categorised between their usage in the back-end and front-end.

Back-end Tools and Coding Practices

Integrated Development Environment (IDE): JetBrains Rider was used as the IDE for the back-end development. This is a specialised IDE developed specifically by JetBrains to work with .NET technologies.

Code Formatting and Style: Rider provides automated code formatting and opinionated styling rules for different aspects of the codebase such as naming conventions or spacing of parenthesis. These were used to improve consistency and code readability.

Documentation: Standardised code documentation was used for each function to take advantage of Rider's method documentation features. This was used to describe the purpose and functionality provided by different methods in the application to then be referenced by other developers when maintaining or extending the application.

Front-end Tools and Coding Practices

Integrated Development Environment (IDE): Visual Studio Code was used for the front-end development. This is a popular IDE for web development and provides many features to improve productivity and code quality.

Code Formatting: Prettier was used to provide automated code formatting of the front-end. It is an opinionated code formatter that ensures all code abides by a consistent formatting style.

Code Style and Quality: ESLint was configured using its default settings to provide a ruleset for code quality and code style. This was used to ensure that the codebase was of high quality and followed the best practices for a Typescript application.

Componentisation: The front-end was developed in line with best practices for React development. This included the use of reusable components wherever possible to reduce code duplication and improve maintainability. This also improved the developer experience as generic components could be used easily across the application for a consistent user interface.

4.1.2 Implementation Details

The application consists of 2 main components using multiple different services from AWS and MongoDB to perform the necessary functionalities. Within the back-end and the front-end components, 3 key modules integrate to complete the application. The authentication module, user interaction module and automated report generation module. Each of these modules is responsible for a different aspect of the application and interacts with the other modules to provide the necessary functionality. A detailed description of each module is provided below along with descriptions of the general front-end and back-end components.

General Front-end Component

The front-end component of the system is responsible for all user interaction and display logic. It is a Typescript application using the React framework composed of multiple pages and components to expose the functionality of the system to the user. The front-end component uses the Apollo Client package to handle interaction with the back-end GraphQL API. This library provides many tools to assist with interacting with a GraphQL API including the automated generation of hooks and types as well as state management and effective caching. The Material-UI library was used to provide a set of pre-built components that could be used to quickly develop the user interface of the application. The application was built using the Vite build tool which is a fast and efficient solution for building react applications. It also allows the application to be served in a development environment with hot reloading to quickly deploy changes as they are made. NPM was used as the package manager while developing, handling the installation and management of all packages and dependencies in the development environment.

General Back-end Component

The back-end component of the system is responsible for handling all business and data access logic in the system. It is a C# application using the ASP.NET Core framework. The application is structured using a Controller-Service-Repository (CSR) architecture. This architecture uses a layered approach to the separation of concerns for better testability and maintainability.

Controller Layer: The controller layer defines the endpoints of the application. In the back-end component, this layer consists of GraphQL query and mutation classes exposing the functionality of the application.

Service Layer: The service layer is responsible for handling the business logic of the application. The layer is composed of a set of services that are available for the controller layer to call.

Repository Layer: The repository layer is responsible for handling the data access and manipulation logic. This layer consists of a set of repository classes that abstract the communication with the database from the service layer. This allows for a more flexible, maintainable and testable application as changes to the database and data access logic are separated from the business logic.

Model Layer: Finally, a model layer defines the data structures used in the application.

The application uses the Hot Chocolate GraphQL framework to create and expose a GraphQL API to the front-end. Hot Chocolate is a popular open-source technology used to build GraphQL APIs in .NET. It is a powerful tool with many features that enable clean, functional and maintainable code. The application uses the annotation-based approach, using it for the definition of the endpoints with queries and mutations as well as the handling of authorisation. The back-end component also uses the Hot Chocolate MongoDB package to enable communication with and management of the database. Finally, the Fluent Validation package is used to handle the validation of incoming request objects for the application providing a consistent and maintainable way to validate request data.

Authentication Module

The authentication module is responsible for handling the user authentication and authorisation in the application. The module is composed of 2 main components, the front-end authentication component and the back-end authentication component.

The front-end component is responsible for handling the user login and registration process in the application. These responsibilities are shared between the AWS Amplify UI library's Authenticator component for logging in and forgotten password processes and the Super Admin interface for user registration and role assignment. For the login functionality, the 'Authenticator' React component provided by the 'aws-amplify/ui' library was used. This component provides a pre-built implementation for logging into an application secured by an AWS Cognito User Pool. The component is given the configuration for the user pool and provides a login form, a forgotten password flow and if configured a sign-up flow. The component handles interaction with AWS Cognito to authenticate the user and retrieve the appropriate tokens as well as automatically storing the tokens in the browser's local storage. Additionally, the component provides an object holding the user's information and a callback to sign the user out. The library also provides hooks that can be used to retrieve custom user attributes, this functionality was used to retrieve the user's role for authorisation purposes in the application. The Super Admin interface was developed to allow the Super Admin user to register new users and assign them roles in the application. The current purpose of the application does not require the user to be able to register themselves so this functionality was not included in this iteration of the project. The Super Admin interface was a custom-developed set of components that provide a table of users from which a user can be created, edited or removed. The interface interacts with the back-end to perform these actions and to retrieve the necessary data to display to the user.

The back-end component of the authentication module is mainly responsible for handling the user authentication and authorisation and actioning requests to create, edit or remove users in the application. The back-end component is composed of 2 main parts, the user authentication and authorisation middleware and the user management service. The user authentication and authorisation middleware is responsible for checking the user's token and role before allowing them to access an endpoint. The application uses the standard Microsoft authentication middleware to handle the JWT token provided in the Authentication header of each request. The middleware checks with AWS Cognito that the token is valid before continuing to process the request. For authorisation, the application makes use of Hot Chocolate authorisation annotations to denote the required roles for access to each endpoint. A mapping of roles to policies is made available to Hot Chocolate and these policies can then be applied to an endpoint to restrict access. Hot Chocolate's middleware will access the user's role from the token and compare it to the policy for the endpoint. If the user's role is not sufficient to access the endpoint, the request will be rejected. The user management service is responsible for actioning any user creation, edit or removal requests made by the Super Admin user. The user management service interacts with the MongoDB database through the Hot Chocolate MongoDB package and the AWS Cognito User Pool through the AWS SDK to perform these actions.

User Interaction Module

The user interaction module is responsible for managing all user interactions within the application. This module is the most extensive, encompassing the majority of front-end components that present information to the user and facilitate user interactions. It also includes the back-end components that process the requests generated by these interactions.

The front-end component is responsible for the presentation of the data to the user and handling any user interaction with that data. This can be broken down into the display and interaction with reports, projects and users. The components that handle this interaction are all custom-built using the Material UI library for a clean, consistent interface that can be created quickly. All forms are created using the Formik library for handling form state and yup for validation. The components make use of the hooks and types generated by the Apollo Client to interact with the back-end GraphQL API. The use of Apollo Client to generate hooks and types means the system is easier to maintain and the front-end application can be kept in sync when changes are made in the back-end.

The back-end component is responsible for handling the requests generated by the interaction from the user. For this module, these mostly consist of Create-Read-Update-Delete (CRUD) operations on the data stored in the database. The back-end application exposes query and mutation endpoints using the Hot Chocolate framework to allow the front-end to enable interaction with the data. This module makes full use of the CSR architecture to separate control, business and data access logic for better testability and maintainability. When a request is received, the authentication module will determine the user's permission to access the endpoint before the request is passed to the controller layer. The service layer will then perform the necessary business logic utilising the abstracted access to the data provided by the repository layer. The repository layer will interact with the database to perform the required CRUD operations and return the data to the service layer. The service layer will then return the data to the controller layer and finally back to the front-end application for display. The use of the Hot Chocolate framework allows for complex query logic to be abstracted from the controller layer, ensuring the application is maintainable and testable. Hot Chocolate provides access to GraphQL resolvers that allow for complex query logic to be defined in a single place and reused across multiple endpoints. For example, if a specific object should include a computed field not present on the model, a resolver can be used to compute and add this field to the object before it is returned to the front-end. Resolvers can also be used to handle complex query logic such as filtering, sorting and pagination. Hot Chocolate provides built-in functionality to handle these operations and abstract them from the controller layer. An annotation can be included for each operation and custom logic can be provided through specially annotated classes. This allows for the application to be easily extended and maintained with complex query logic re-used across multiple endpoints.

Automated Report Generation Module

The automated report generation module is responsible for handling the generation of reports through a connection to the Clockify API. The module is composed of 2 main components, the Clockify service and the scheduled report generation trigger.

The Clockify service is responsible for connecting the back-end application to the Clockify API, retrieving the necessary data and creating the reports. The application exposes an endpoint to trigger the report generation process through a REST controller, once this endpoint is hit, the controller calls the Clockify service to start the report generation process. The Clockify service interacts with the Clockify API to retrieve the necessary data to create the reports. The process of calls is quite complicated as the Clockify API is not very well designed. The service must first retrieve all the workspaces in the Clockify account, and then select the correct workspace to retrieve its ID. Once the workspace ID is retrieved, it must be used in all subsequent calls to access the data in the workspace. The service then retrieves all active users in the workspace to individually request their time entries logged in the last week. The final call to the Clockify API requests all active projects in the workspace. The service can now loop through each retrieved project, filtering the time entries by the project ID and calculating the required metrics for the reports. The service then stores the report in the database via the project repository.

The second component of the automated report generation module is the scheduled report generation trigger. This component uses a mix of AWS services to schedule and trigger the report generation process in the application. The component is composed of an AWS Lambda function and an AWS Eventbridge event. The Lambda function is a C# function that makes a call to the back-end application's report endpoint that triggers the report generation process. The Lambda function is triggered by an Eventbridge event that is scheduled using a cron expression defining a trigger for the function every Wednesday at 11:59 pm. The Eventbridge event was created using the AWS console and the Lambda function was developed in Rider as an extra project in the main repository. The Lambda function is deployed to AWS using the AWS CLI. It is developed using the AWS Lambda annotations framework which in the future could be used as part of a CI/CD process to automatically deploy the function to AWS when changes are made to the repository.

4.1.3 Development Challenges

Throughout the development of the project, various challenges arose that required resolution to ensure the application functioned correctly. Each of the three key modules presented distinct issues that needed to be addressed to achieve a successful implementation.

Authentication Module

The authentication module presented multiple challenges in its implementation. The first presented itself in the creation of an AWS Cognito User Pool. At this point, I had no experience with AWS Cognito and very little experience in working with standard authentication workflows. This made it challenging to create the initial user pool and configure it correctly to store the additional information I needed for the application. To overcome this challenge, I spent time reading the AWS documentation on Cognito and watching tutorials on how to set up a user pool. This allowed me to understand the basic concepts of Cognito and how to configure it to store the necessary information.

The second challenge presented itself when it came to setting up the login page for the front-end application and understanding how this worked with authentication in the back-end application. I had worked with a

component-based system before and had dealt with authentication problems but always in the context of a semi-complete system. This was the first time I had to set up the entire system from scratch and ensure that the front-end and back-end were correctly communicating. To overcome this challenge, I spoke with my supervisor and the team member most experienced with infrastructure about the best way to handle the authentication process. I was directed to a good example to follow and this helped me to understand the process and implement it correctly.

The next challenge occurred when configuring the back-end application middleware to handle the user's token and role for authentication and authorisation. I used the Microsoft authentication middleware to handle the JWT token provided in the Authentication header of each request. This middleware checks with AWS Cognito that the token is valid before continuing to process the request. I had never worked with the authentication middleware before so again needed to speak to someone more experienced in infrastructure about how to set this up. I was directed to an example of another project that used a similar set of technologies and this helped me to understand how to set up the middleware correctly. Using Hot Chocolate annotations for authorisation was also a new concept to me. I had used the annotations for authorisation of endpoints before but only to extend a configured application and not to set up from scratch. I spent time reading the Hot Chocolate documentation on authorisation and watching tutorials on how to set up authorisation in Hot Chocolate as well as referring to the example project I was given by Jaegersoft to understand how to set up the authorisation correctly.

The final challenge came when trying to customise the login page for the front-end application to be more in line with the styling of the rest of the application. The AWS Amplify UI library's Authenticator component provides a pre-built implementation for logging into an application secured by an AWS Cognito User Pool. The Amplify UI library is very opinionated and does not provide many customisation options. To overcome this challenge, I spent time reading the documentation and searching for solutions other developers had found. I ended up having to use CSS styling to override the default classes that the authenticator component uses for styling to achieve the desired look.

User Interaction Module

The user interaction module was more aligned with the type of work I had experience with developing using C# and React but it still presented some challenges. The first challenge presented itself when trying to set up the Apollo Client to automatically generate hooks and types for the GraphQL API. I followed the documentation provided by Apollo Client to set up the tool but ran into issues with generating the hooks. I ended up having to refer to a similar project I had worked on before and copying the configuration from there to get the hooks to generate correctly.

The second challenge I faced was using yup for validation of my forms on the front end. I have experience using yup on simple forms but ran into some issues with a specific form that required more complex validation. I had to refer to the yup documentation and examples to understand how to set up the validation correctly. Overall this module presented fewer challenges than the authentication module but still required some research and problem solving to overcome.

Automated Report Generation Module

The automated report generation module presented the most complex challenges during the implementation of the project. The first challenge encountered involved the use of the Clockify API. Clockify provides a REST API to access the application programmatically. The API is not designed to access the data in the way this project requires. The intended use of the API is to fulfil a similar role as Clockify with a different front-end. This did not include the functionality to access all time entries for all users in a workspace or a similar query. To overcome this challenge, I had to spend time reading the Clockify API documentation. I eventually found a workaround where I could retrieve all users in the workspace and then retrieve all time entries for each user. This was not ideal as it required multiple calls to the API and was not very efficient but it was the only way to access the data I needed.

The next challenge presented itself was the design of the scheduling for the report generation process. I initially planned to use Hangfire, a .NET library for scheduling background tasks, to trigger the report generation process. However, I realised after some discussion with my supervisor that this was inefficient when compared to other options available with AWS. I had a decision to use AWS Lambda to trigger the process, Hangfire to trigger the process or AWS Lambda to perform the process. I decided to use AWS Lambda to trigger the process as it was the most efficient and cost-effective solution for the proof of concept application. I had already done the work for the generation process in the application and exposed an endpoint so it was a simple change to make. Ideally, I would abstract this functionality from the application and use the Lambda function to perform the process but this was not possible due to time constraints.

The final challenge encountered was the deployment of the Lambda function. I had experience working with Lambda in the past and using CI/CD to deploy the functions but only in a repository that was dedicated to lambda functions. I had never deployed a lambda function from a repository that contained a full-stack application. I spent some time researching how to deploy the function and found that the most suitable way to deploy the function without an existing CI/CD pipeline for the repository was to use the AWS CLI. I followed the AWS documentation on deploying a lambda function using the CLI and was able to deploy the function successfully. I would have liked to use a CI/CD pipeline to deploy the function but this approach was not suitable as there was no existing pipeline and I did not have the time to set one up. Ideally, I would set up an action in the GitHub repository that would handle CI/CD for the entire application including the Lambda function.

Deployment Challenges

I found the deployment of the application as possibly the most challenging aspect of its implementation. I have been involved in the deployment of a few applications but not solely responsible for the deployment of an application. The first problem I found was that AWS Amplify automatically provides a certificate for the front-end application whereas AWS Elastic Beanstalk does not. This resulted in all communication between the 2 being blocked due to a 'Mixed Content' error. To resolve this a custom domain needed to be set up for the back-end application and a certificate needed to be obtained. However, the proof of concept deployment of the application did not use a load balancer as it was only a single instance deployed. This meant the

certificate could not be assigned to the application easily. To work around this, I had to set up a CloudFront distribution in front of the Elastic Beanstalk application and assign the certificate to the distribution. This means that communication between the front-end and the Cloudfront distribution was secure over HTTPS and the distribution could communicate with the back-end over HTTP. This was not ideal as it added an extra layer of complexity to the deployment but it was the only way to get the application to communicate securely without needing to pay for an application load balancer.

4.2 Project Evaluation

The project evaluation section will provide details on the evaluation process of the proof of concept application. It is important to note that due to the nature of the project as a proof of concept application, the evaluation of the project's success heavily influences the results of the research. Due to this, this section will include information on the evaluation methods used to assess the success of the application and the results of any methods not involved in the research. Any evaluation method that was used as part of the research of the application will have its results presented and discussed in section 5.1. The limitations of the project will also be discussed in this section. This will include any limitations of the application itself. Finally, the future improvements that can be made to the project will be discussed. This will include any potential improvements to the application that could be made in the future.

4.2.1 Unit & Integration Testing

The first form of evaluation involved the use of unit testing. This was an ongoing process in the implementation process of the application. Unit tests were written to ensure the application's units reliably performed their expected function and the presence of defects was minimised. The unit tests were written using the NUnit testing framework with NSubstitute used as the mocking framework and Fluent Assertions used for assertions. The tests were written to cover any business logic that provided risk to the application, any critical functions or any substantially complex functions. The tests were created towards the end of the development phase before the application was deployed to ensure the application was functioning as expected. They were also used for regression testing to ensure that new changes did not introduce defects in completed functionality.

Integration testing was performed after the completion of all unit testing. These tests utilised the same functionality but looked to test the interaction between the different components of the application. The integration tests were written to simulate interaction from the front-end to the back-end and back again. The tests triggered endpoints of the application with different data and checked the expected response was returned and the correct data manipulation was performed. The integration tests used the same technologies as the unit tests as well as the Mongo2Go package to provide a temporary in-memory database for the tests to interact with.

The tests were run using Rider's internal test runner and the test coverage was calculated using Rider's internal coverage tool. The quality, results and coverage of the tests can be used to effectively evaluate the reliability of the application. Figure 4.1 shows the results of the tests run on the back-end application.

Symbol	Coverage (%)	Uncovered/Total Stmt.
Total	92%	26/345
API	92%	26/345
{} Backend	92%	26/345
{} User	97%	3/107
{} Validators	100%	0/13
CreateUserValidator	100%	0/13
CreateUserValidator()	100%	0/9
BeValidJaegersoftEmail(string, CancellationToken)	100%	0/4
UserType		0/0
UserService	96%	3/83
UserService(IAmazonCognitoIdentityProvider, IMongoClient, IConfiguration, IUserRepository, IProjectRepository)	100%	0/8
UpdateUser(string, string, string, string, UserRole, List<string>)	96%	1/27
RemoveUser(string)	95%	1/21
CreateUser(string, string, string, UserRole, List<string>)	96%	1/27
UserMutations	100%	0/11
UpdateUser(string, CreateUserDto, IUserService, CreateUserValidator)	100%	0/4
RemoveUser(string, IUserService)	100%	0/3
CreateUser(CreateUserDto, IUserService, CreateUserValidator)	100%	0/4
{} Report	100%	0/41
{} Validators	100%	0/7
ApproveReportValidator	100%	0/7
ApproveReportValidator()	100%	0/7
own coverage	100%	0/6
(string)	100%	0/1
ReportType		0/0
ReportService	100%	0/26
ReportService(IReportRepository, IProjectRepository)	100%	0/5
ApproveReport(string, string, ProjectHealth, CancellationToken)	100%	0/21
ReportMutations	100%	0/8
ApproveReport(ApproveReportDto, CancellationToken, IReportService, ApproveReportValidator)	100%	0/8
MoveNext()	100%	0/8
{} Project	100%	0/68
{} Validators	100%	0/9
CreateProjectValidator	100%	0/9
CreateProjectValidator()	100%	0/9
ProjectType		0/0
ProjectService	100%	0/39
ProjectService(IProjectRepository, IUserRepository)	100%	0/5
UpdateProject(string, string, DateTime, string, double, string, double, CancellationToken)	100%	0/14
RemoveProjects(List<string>, CancellationToken)	100%	0/8
GetProjectsByUserId(string)	100%	0/7
CreateProject(string, DateTime, string, double, ProjectStatus, string, double, CancellationToken)	100%	0/5
ProjectMutations	100%	0/20
UpdateProject(string, CreateProjectDto, CancellationToken, IProjectService, CreateProjectValidator)	100%	0/8
RemoveProject(List<string>, CancellationToken, IProjectService)	100%	0/7
CreateProject(CreateProjectDto, CancellationToken, IProjectService, CreateProjectValidator)	100%	0/5
ClockifyProjectType		0/0

FIGURE 4.1: Unit Test Coverage Report

The coverage report shows a 92% coverage of the back-end application. This is a good level of coverage for the application and indicates that the application's critical functionality is well-tested. In addition to the high test coverage, the coverage report shows all tests as passing. The quality and coverage of the tests indicate that the application is reliable and the risk of defects is minimised.

4.2.2 Usability Testing

Usability testing was used to evaluate the usability of the application. A usability testing session was conducted as discussed in section 3.2.1. The session was used to collect data useful to analyse the usability of the application and evaluate its performance with a representation of the end user. The usability session consisted of 3 related sections.

Observational Metrics

The first aspect of the usability testing section involved the collection of observational metrics while users completed tasks designed to simulate normal use of the application. As discussed in section 3.2, the observational

metrics included the time taken to complete each task, the number of clicks made and any errors made by the participants. The tasks used to simulate the normal use of the application were performed sequentially with each task following on from the next to simulate a user's journey through the application. The tasks were as follows:

Task 1: Can you tell me what projects you have been assigned?

Task 2: Can you tell me the date of the last report the client can see?

Task 3: In the latest report, what was the health of the project?

Task 4: Can you tell me the date of the latest report awaiting approval?

Task 5: Can you approve the latest report?

The observational metrics were collected manually and recorded in Microsoft Excel. This data was then analysed as discussed in section 3.2.3 to evaluate the complexity and usability of the application. This evaluation was used to assess the application's ability to meet the needs of the client and the usability of the application in general. The results will be presented and discussed in section 5.1.

SUS Questionnaire Data

Next, the participants were asked to complete a System Usability Scale (SUS) questionnaire as discussed in section 3.2. The SUS questionnaire was used to gather feedback on the application's suitability for status reporting and to assess the overall usability of the application. The questionnaire followed the standard format of the SUS questionnaire of 10 statements with a 5-point Likert scale, each statement alternating in positivity and negativity. The statements used were as follows:

Q1: I think that our clients would like to use this system frequently.

Q2: I found the system unnecessarily complex.

Q3: I thought the system was easy to use.

Q4: I think that I would need the support of a technical person to be able to use this system.

Q5: I found the various functions in this system were well integrated.

Q6: I thought there was too much inconsistency in this system.

Q7: I would imagine that most people would learn to use this system very quickly.

Q8: I found the system very cumbersome to use.

Q9: I felt very confident using the system.

Q10: I needed to learn a lot of things before I could get going with this system.

The average SUS score was calculated based on the results of the SUS survey and the results of individual questions used as feedback on specific elements of the application. The results were used to evaluate the application's usability and to gather feedback on the application's suitability for status reporting. The application would be considered usable with an average SUS score of 75. This is a high score representing an application with high usability standards. The results of the evaluation found that the developed application reached the success criteria. The results will be presented and discussed in section 5.1.

Discussion Data

The final aspect of the usability testing section involved the collection of discussion data. The discussion data was collected through a semi-structured interview with the participants. The interview was used to gather the participant's opinions and perspectives on the application. A list of questions was prepared to guide the discussion and ensure all necessary information was collected. The questions were as follows:

- Q1:** What did you think about the application navigation?
- Q2:** Do you have any thoughts on how the report information was displayed?
- Q3:** What about the ease of getting to the report information?
- Q4:** What would you do to change the application to make it more effective? Eg. The layout, etc.
- Q5:** How do you think it would affect the current workflow for our reporting?
- Q6:** Were there any points in the previous scenarios, where you felt confused or unsure about what to do next?
- Q7:** Did you encounter any issues or bugs while using the application? If so, please describe them.
- Q8:** How well do you think the application integrates with Clockify?
- Q9:** Were there any features that you felt were missing or could be improved?
- Q10:** What features did you find the most useful, and why?
- Q11:** Do you have any additional feedback or suggestions for improvement?

The discussion data was recorded manually and analysed as discussed in section 3.2.3 to evaluate the application's suitability for status reporting and the user's experience with the application. The application would be considered as meeting the client's standard if the discussion data showed at least 75% of responses could be considered positive. The responses collected found that the application reached this success criteria. The results will be presented and discussed in section 5.1.

4.2.3 User Acceptance Testing

A user acceptance testing period was used to evaluate the application's ability to meet the client's needs, measure the effectiveness of the application and gather feedback on the application's performance. The user acceptance testing period was conducted over 2 weeks with 2 Jaegersoft employees. The period involved 2 main components, concurrent usage assessment and client feedback.

Concurrent Usage Assessment

The concurrent usage assessment was conducted as discussed in section 3.2. The assessment was used to evaluate the application's ability to replace the manual reporting process and to gather feedback on the application's performance. This included the time taken to generate the reports, the cost of generating the reports and the user's opinion on the application's effectiveness. The application would be considered a possible replacement for the manual process if it achieved a 50% reduction in both time efficiency and cost-effectiveness. The results of the evaluation found that the application met these requirements and can be considered a possible replacement for the manual process. The results of the concurrent usage assessment will be presented and discussed in section 5.2.

Client Feedback

The client feedback was collected throughout the user acceptance testing period in an unstructured manner. The feedback was used to gather the client's opinion on the application's effectiveness and efficiency as well as communicate any defects found during the usage period. The feedback was recorded manually and documented mostly as quotes from the client. The defects found during the usage period were recorded using Jira tickets marked as bugs.

During the UAT period, 8 defects were found and recorded as bugs. 1 of these was a major defect that affected the application's ability to generate reports. The defect was fixed during the testing period to ensure the application was functioning correctly. The remaining 7 defects were minor defects that did not affect the application's functionality only the cosmetics of its display. The defects were fixed after the testing period to ensure the application was of high quality. The feedback from the client was mostly positive during this period, with the client expressing satisfaction with the application's performance and effectiveness. There were a few suggestions for improvement that were recorded and will be part of the future improvements of the application. The positive results of the client feedback suggest that the application is effective and efficient in its current state and meets the client's needs.

4.2.4 Evaluation of Critical Success Criteria

The critical success criteria outlined in the revised project plan were used to evaluate the success of the application. The success criteria were as follows:

- SC1:** The application must reduce the time spent calculating metrics for weekly progress reports by 60%.
- SC2:** The application must provide a 30% reduction in overall costs associated with reporting.
- SC3:** The project manager users and client users must be able to access the application and generate reports with no more than 3 clicks.
- SC4:** The application must be able to load up to 10 reports in under 5 seconds.

The success criteria were used to evaluate the application's performance and usability and judge whether the application was successful in meeting its intended purpose. The purpose of the application was to provide

a more efficient and effective way to generate weekly progress reports for the client. Each of these success criteria were tested upon completion of the application to ensure the application was successful. SC1 and 2 were evaluated through the UAT period as part of the concurrent usage assessment and their results will be presented in section 5.2. SC3 was evaluated through the use of the application and was found to have been met. SC4 was also evaluated through the use of the application, the time to load 10 reports was recorded at 1.8 seconds. The evaluation found that all of the success criteria were met by the application and the application was successful in meeting its intended purpose.

4.2.5 Limitations

Although the application was successful in meeting its intended purpose and the critical success criteria, there were some limitations to the application in terms of functionality. The first limitation seen in the application was the design of the user interface, its lack of Jaegersoft branding and modern feel for the user. The proof of concept application focused more on the functionality and usability of the application than its looks and branding. As such, the interface would need to be redesigned to meet the client's branding and modern design standards.

The application uses Clockify as its source of data for generating the reports. This is an external dependency on an application that is not always reliable. Unfortunately, this limitation could not be avoided as it is an existing business process for Jaegersoft. This could mean there is a case where the report generation process is triggered but the Clockify API is not available and the process fails. This could be mitigated by adding a retry mechanism to the report generation process to ensure the report is generated when the Clockify API is available.

The application's users need to be created and managed manually along with the projects. This necessitates a Super Admin user with the ability to create and manage users and projects through the system UI. This is a limitation as it requires manual intervention to ensure all application users have access to their required resources and that the application is up to date with the client's current projects. This could be mitigated by using tags on projects in Clockify to tag projects to exist in the application and automatically create them when the report generation process is triggered. The users could also be managed through a separate client management system that would be integrated with the application but unfortunately, that is outside the scope of this project.

4.2.6 Future Improvements

Improvements to the application in the near future would mainly be based on improving the user experience and bringing in new functionality to the application. The first improvement would be to integrate more of Jaegersoft's branding and upgrade the interface to look more modern and professional. This would involve a complete redesign of the user interface to meet the client's standards and possibly the engagement of a professional designer. The next priority would be to allow feedback from clients on the report page. Currently, the application only allows the client to view the report, this feature would allow clients to comment on the usage of hours or ask questions about the report in the application. This would provide a more interactive

experience for the client and allow for more effective communication between Jaegersoft and the client.

Future extensions of this project would focus on the integration of Jira data and support for fixed-price contracts. Currently, the application integrates only with Clockify as a data source for report information, the addition of Jira data would provide a more complete picture of the project's progress and how time is being used. The application also only supports retainer-based contracts where a client purchases a grouping of work hours from Jaegersoft and the progress of the project is based on the usage of those hours. A fixed price contract is where Jaegersoft engages with the client in a lengthy design process to scope out an entire project and the client purchases the development of the scoped project. The application does not currently support this type of contract as it was out of scope for this project but it would be a valuable addition to the application.

4.3 Summary

The implementation of the proof of concept application was discussed along with the methods used to evaluate the success of the application. The application contained 3 key modules, the authentication module, the user interaction module and the automated report generation module. Each module was discussed in detail along with the challenges faced during the implementation of the project. The evaluation of the application was discussed in detail with the results of any evaluation not used in the research discussed. The evaluation found that through the use of multiple forms of automated and manual testing, the application was successful in meeting its intended purpose and critical success criteria. The limitations of the application were discussed along with the future improvements that could be made to the application. The limitations included the design of the user interface, the reliance on the Clockify API and the manual management of users and projects. The future improvements included the integration of Jira data, support for fixed price contracts and the addition of client feedback to the report page. The results of the usability testing and the user acceptance testing will be presented and discussed in the results section.

5

Results and Discussion

The results and analysis section will lay out the results of the research and analyse the findings to determine the conclusions that can be drawn. The results of the research will be used to answer the research objectives and determine future research in the area.

5.1 Usability Testing Results

The usability testing session was conducted as discussed in section 3.2.1. The session was used to collect both quantitative and qualitative data on the usability of the application through three different methods. The goal of the usability testing data is to provide answers for research objectives 1 and 4. Is an automated solution for report generation using Clockify data suitable for status reporting and can an automated solution meet the needs of the client in the context of status reporting? The results from the usability testing will provide part of the data necessary to judge whether the developed application was successful in meeting the needs of the client and if not, find where the application fell short.

5.1.1 Observational Metrics

Observational metrics were collected as discussed in section 3.2. 5 different testing participants were asked to perform a set of tasks designed to simulate the normal use of the application while the time taken to complete the tasks, the number of clicks made and the number of errors were recorded. The results of the observational metrics are presented in Table 5.1.

Scenario	Tester	Time (s)	Clicks	Errors
1	1	4	1	0
2	1	21	2	1
3	1	6	1	0
4	1	46	15	5
5	1	15	5	1
1	2	4	1	0
2	2	16	2	0
3	2	5	2	1
4	2	6	1	0
5	2	17	6	1
1	3	4	1	0
2	3	11	1	0
3	3	2	1	0
4	3	2	1	0
5	3	6	2	0
1	4	5	1	0
2	4	18	2	1
3	4	7	1	0
4	4	4	1	0
5	4	10	6	1
1	5	3	1	0
2	5	17	2	1
3	5	6	1	0
4	5	3	1	0
5	5	8	2	1

TABLE 5.1: Scenario Testing Results

Each task followed on from the previous and required different interactions so the variance in time taken

and number of clicks between tasks can be expected. Figure 5.1 shows the time taken to complete each task for each tester. We can see for the most part that the time taken to complete each task is consistent between testers except for a large spike on task 4 for the first tester. During testing, this tester became confused with the question and spent considerably longer on this task than other testers. Due to this abnormal result, the tester's data for that task was removed from the analysis.

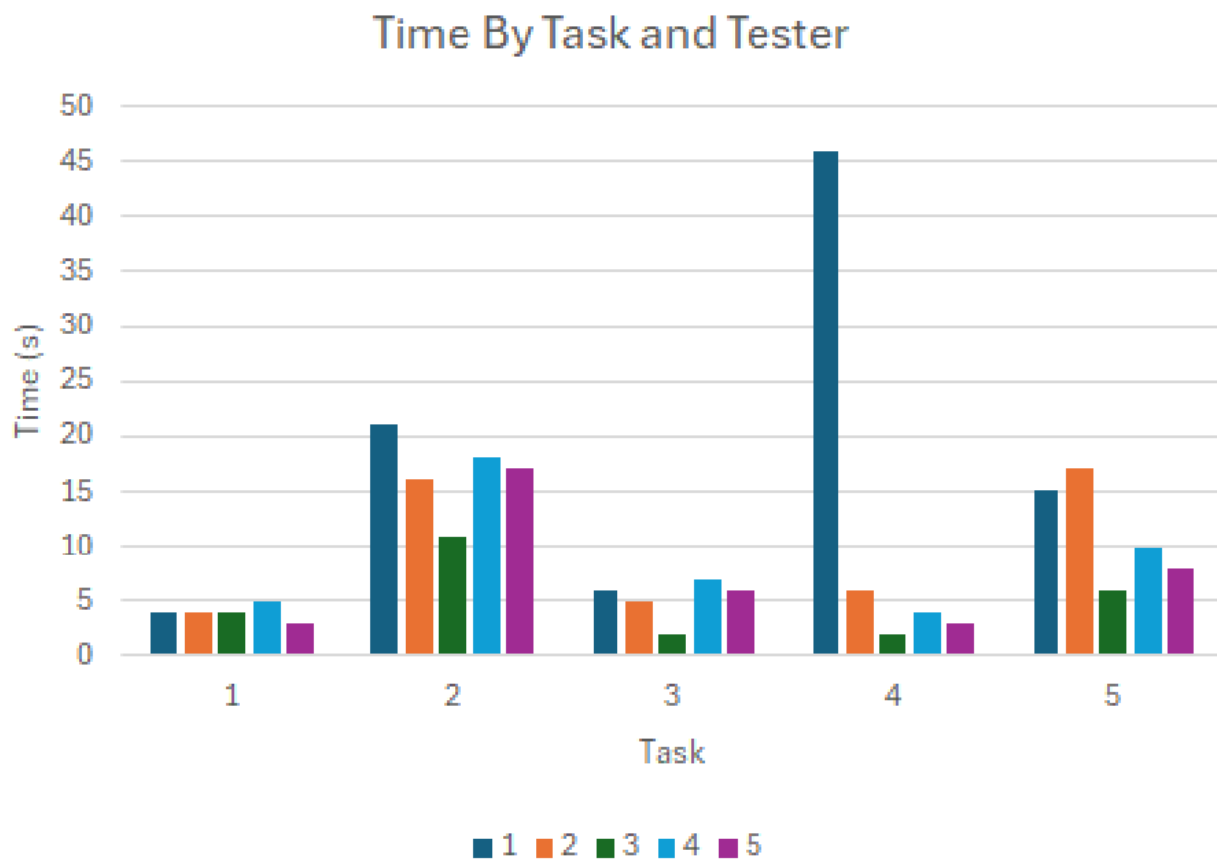


FIGURE 5.1: Time Taken to Complete Tasks by Tester

Once the data has been pre-processed, meaning can be drawn from the data. The first useful metric from this data is the percentage of clicks that were errors. This metric is useful as a measure of the complexity of the interface or task. When a task contains a high percentage of errors to clicks, it can be inferred that the task is complex and difficult to complete. The results of this metric are presented in Figure 5.2. The results show that task 2 had the highest percentage of errors to clicks at 40% and tasks 1 and 4 had the lowest at 0%. This indicates that task 2 was the most complex task to complete for users. This difference could be exaggerated by the task requiring very few clicks to complete. In this situation, a single error would result in a high percentage of errors to clicks. The metric is still useful when it is used to compare against the other tasks that all require very few clicks to complete but a high percentage of errors should not be considered a major issue.

40% of clicks as errors is a high percentage and indicates that the current design of the interface or flow of the task needs to be improved to make the task easier to complete. It is worth noting that these tests were conducted on users who had not used the application before and were not familiar with the interface. This high percentage of clicks as errors could also be due to the unfamiliarity of the users with the interface and may suggest a lack of appropriate affordances to guide new users. A successful status reporting application should be easy to use and require minimal training for clients to use effectively. The high percentage of errors in task 2 suggests that the interface needs to be improved before the application can be considered suitable for a client-facing progress reporting application.

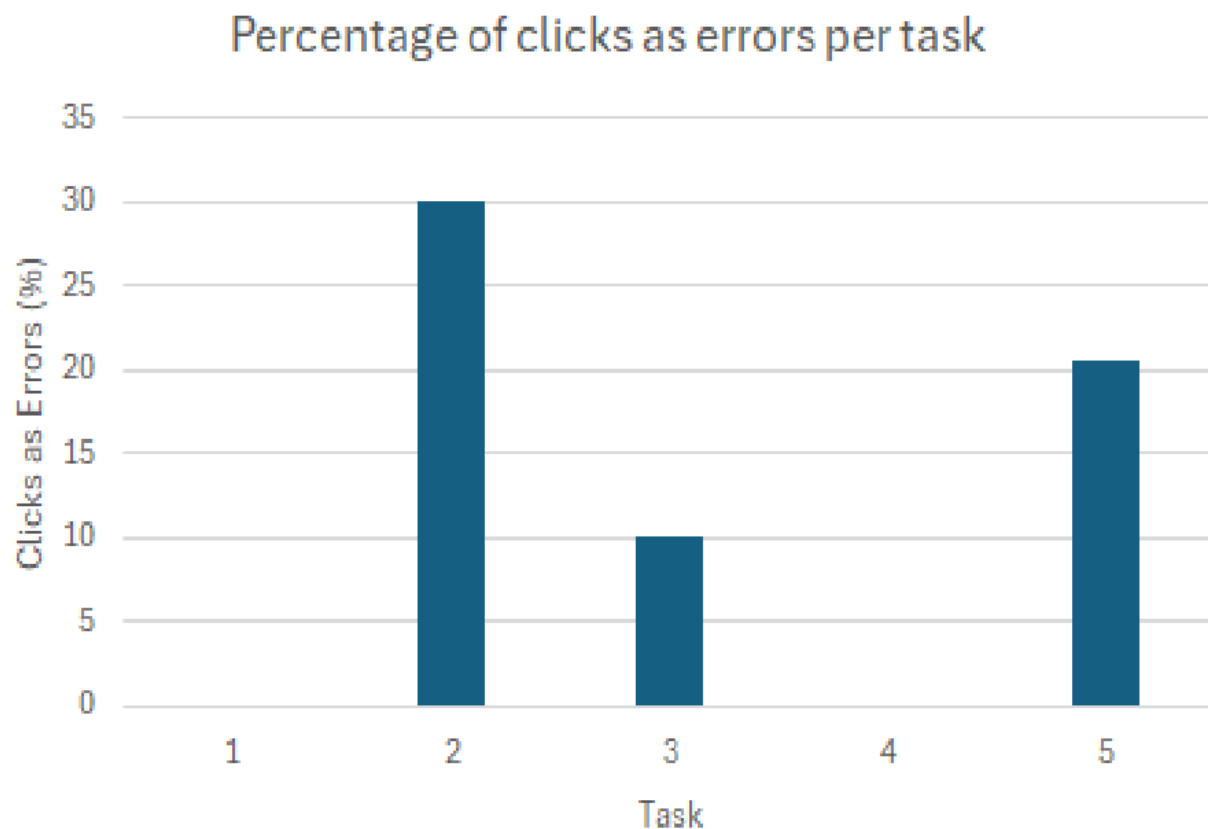


FIGURE 5.2: Percentage of Clicks as Errors by Task

The average time taken per click can also be extracted from this data. This metric can also be used to suggest complexity in the interface. This metric is more related to the ability of a user to understand the information displayed in the interface as it measures how long between clicks a user takes to think. A major flaw in this metric that needs to be accounted for is each task may require different interactions with the back-end application and the metric risks being affected by the load times for the interaction between the front-end and back-end applications. The testing session was conducted in a location with a high-quality internet connection

without changing the amount of data being loaded from the back-end application. There were no instances where an especially long load time was witnessed so these results can be trusted. The results of this metric are presented in Figure 5.3.

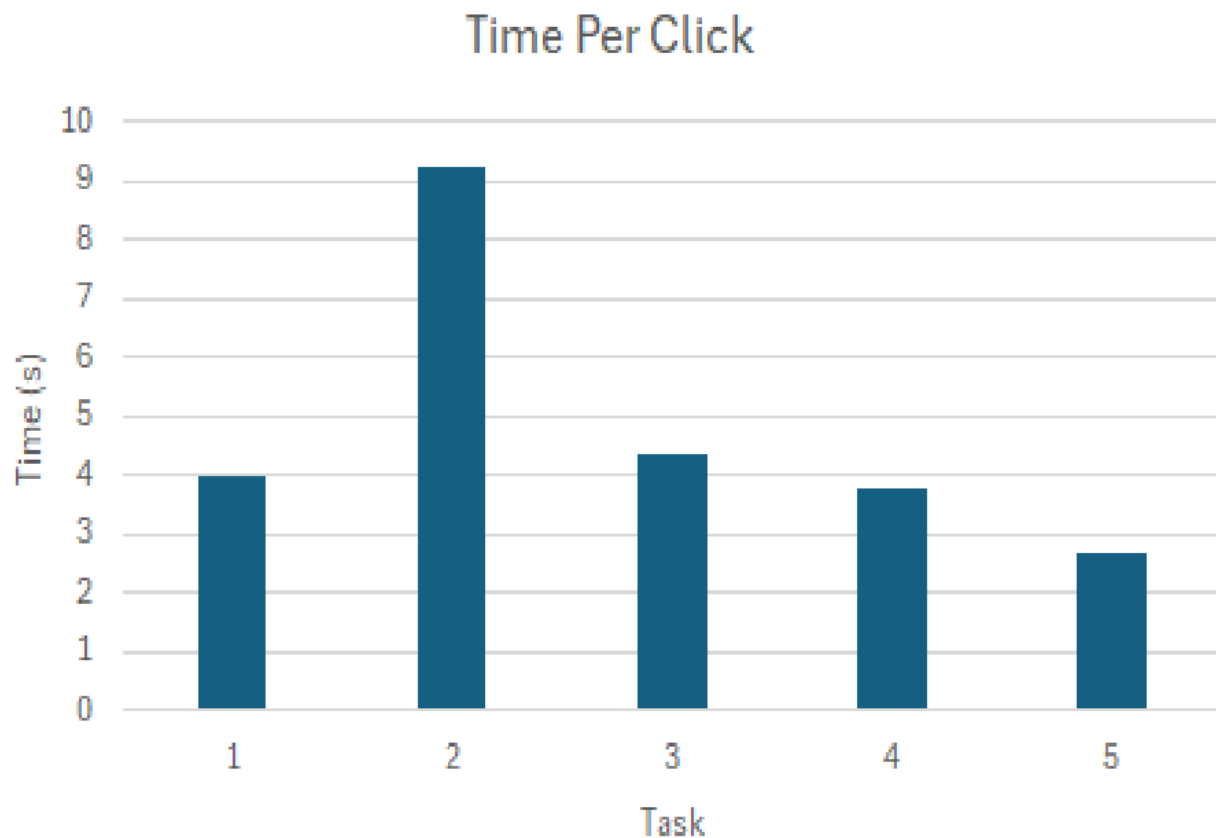


FIGURE 5.3: Time Taken per Click by Task

It can be seen that again task 2 has the highest value with just over 9 seconds per click. This is a very high value considering it is double the next highest value. These results are consistent with the previous metric and suggest that task 2 is the most complex task to complete. The other tasks are all relatively similar in time taken per click with task 5 being the lowest at just under 3 seconds per click. The high value for task 2 suggests that not only are the affordances in the task not appropriate but the way the information is displayed in the interface could be confusing or difficult to understand. Task 2 required the user to find the date of the last report the client can see. The task required navigation from the projects page to the reports page where the user could identify the latest report. The high time taken per click suggests that the users had difficulty understanding the methods for navigation in the application and the information displayed on the reports page. This is a major issue for the application as the ability to quickly find the information required is a critical aspect of a status reporting application. The combination of the high percentage of errors and time per click indicates

that the navigation and information displayed on the projects and reports pages need to be improved before the application can be considered suitable for a client-facing progress reporting application.

Apart from the results for task 2, the application performed well in the observational metrics. The other tasks had a low percentage of errors and time per click values indicating that the tasks were easy to complete and the information was easy to understand. This suggests that an automated solution using Clockify data could be suitable for status reporting and could meet the needs of a client in the context of status reporting.

5.1.2 SUS Questionnaire Data

An SUS questionnaire was conducted during the usability testing session as discussed in section 3.2. The questionnaire was used to gather feedback on the application's suitability for status reporting and to assess the overall usability of the application. The processed results of the SUS questionnaire are presented in Table 5.2. The response for each question has been translated to a score between 0 and 4 as part of the SUS calculation 0 is poor and 4 is excellent. The average scores for each question and each tester are provided along with the average SUS score for the application.

Question	Tester 1	Tester 2	Tester 3	Tester 4	Tester 5	Average
Q1	4	4	4	3	4	3.8
Q2	3	4	4	4	3	3.6
Q3	4	3	4	4	4	3.8
Q4	4	4	4	3	4	3.8
Q5	4	3	3	4	3	3.4
Q6	4	2	3	4	4	3.4
Q7	3	4	4	3	4	3.6
Q8	4	4	4	4	3	3.8
Q9	4	3	4	4	4	3.8
Q10	4	4	4	3	4	3.8
Result	95	87.5	95	90	92.5	92

TABLE 5.2: SUS Questionnaire Results

The results of the SUS questionnaire show that the application was well received by the testers. The average SUS score of 92 is considered to be an excellent score. The results show that the application was considered to be easy to use, well-integrated and easy to learn. This is mostly consistent with the results of the observational metrics. The high average SUS score suggests that the application meets the needs of the client well and is suitable for status reporting.

When considering the individual results of questions in the survey, we can see that the distribution of responses between questions is very consistent with responses ranging between 3.8 and 3.4. The lowest recorded response was in question 6 with a response of 2. This question asked if the user thought there was too much inconsistency in the system. The average for this response was slightly lower than other questions indicating that this was a very slight point of contention for users. This is consistent with the results of the observational metrics that showed possible problems with the user interface when completing task 2.

The high average SUS score suggests that the application performs well in terms of usability and meets the needs of the client. The results of the individual questions show that the application is easy to use, well-integrated and easy to learn. They are also consistent with the results of the observational metrics showing a slight concern with the consistency of some aspects of the system. The results of the SUS questionnaire suggest that an automated solution using Clockify data could be suitable for status reporting and could meet the needs of a client in the context of status reporting.

5.1.3 Discussion Data

A semi-structured interview was conducted with the usability testing participants as part of the usability testing session as discussed in section 3.2. The interview was used to gather the participant's opinions and perspectives on the application. Sentiment analysis was used to analyse the discussion data and draw conclusions on the general perception of the application and its ability to satisfy the client's needs. The results of the sentiment analysis are presented in Table 5.3. The table shows the sentiment of each tester for each question and the average sentiment for each question and tester. Sentiment was depicted as a number from 1 to 3 where 1 was negative, 2 was neutral and 3 was positive.

Question	Tester 1	Tester 2	Tester 3	Tester 4	Tester 5	Average
1	3	3	2	3	3	2.8
2	3	1	2	2	2	2.0
3	3	1	2	2	2	2.0
4	3	3	3	3	3	3.0
5	3	3	3	3	3	3.0
6	3	3	2	3	3	2.8
7	3	3	3	3	3	3.0
8	3	3	3	3	3	3.0
9	3	3	3	3	3	3.0
10	3	3	3	3	3	3.0
11	3	3	3	3	3	3.0
Average	3.0	2.6	2.6	2.8	2.8	2.8

TABLE 5.3: Sentiment Analysis

It can be seen from the results that the application was well received by the testers. The average sentiment for each question was 2.8 which is considered to be a positive sentiment. The results show that the application was considered to be easy to use, well-integrated and easy to learn. The only responses received with a negative sentiment were from tester 2 on questions 2 and 3. These questions asked the user for their opinion on the display of the report information and the ease of getting to the report information. The negative sentiment from tester 2 on these questions is consistent with the results of the observational metrics and the SUS questionnaire that showed a possible issue with the navigation in the application and the display of information. The results of the sentiment analysis suggest that an automated solution using Clockify data could be suitable for status reporting but that this application may need some improvements considered for the interface and navigation before it can be considered suitable for a client-facing progress reporting application.

In addition to the sentiment analysis conducted on the discussion data, thematic analysis was used to identify

common themes in the responses from testing participants. The themes identified were as follows:

Inconsistency: The application has some minor inconsistencies in the way navigation was implemented. It was noted multiple times that clicking a project in the project table would be expected to take the user to the project page but instead took the user to the reports page filtered by that project. This inconsistency was noted as a minor issue by the participants.

Spacing of information: The spacing for information was noticed multiple times by respondents as some fields were too close together or looked cluttered due to poor spacing management. This was noted as a minor issue by the participants.

Priority of bucket information: Multiple participants noted that the bucket information was displayed at the bottom of the report display when it should have been displayed as the first piece of information. This was noted as a moderate issue by the participants.

Split of approved and pending reports: Multiple users noted that splitting the reports into approved and pending reports on different pages was unnecessary and only increased the number of clicks required to find the information for project managers. This was noted as a minor issue by the participants.

Positive impact on the current process: All participants spoke of the application as being a positive addition to the current reporting process and noted that it would improve the speed and convenience of generating reports. This was noted as a major positive by the participants.

The common themes from the feedback data suggest that the application is well received by the participants and that it would be a positive addition to the current reporting process. The feedback data also suggests that there are some minor issues with the interface and navigation that need to be addressed before the application can be considered suitable for a client-facing progress reporting application. These results are consistent with the results of the observational metrics and the SUS questionnaire and suggest that an automated solution using Clockify data could be suitable for status reporting and could meet the needs of a client in the context of status reporting.

5.2 Concurrent Usage Results

The concurrent usage assessment was conducted as discussed in section 3.2.1. The assessment was used to collect quantitative data on the application's performance and efficiency. The goal of the testing was to provide answers for research objectives 2 and 3. Is an automated solution more efficient for the business and is an automated solution more cost-effective for the business? The results from the concurrent usage assessment will provide the data necessary to evaluate whether the developed application was more efficient and cost-effective than the current manual reporting process.

The results of the concurrent usage assessment are presented in Table 5.4. The table shows the time recorded to generate a report for each tester and the method used to generate the report.

	Tester 1	Tester 2
Manual	21	19
Automated	5	5

TABLE 5.4: Comparison of Manual and Automated Reporting Times

The data is depicted graphically in Figure 5.4. The results show that the manual reporting process took an average of 20 minutes to complete while the automated reporting process took an average of 5 minutes to complete. This is a 75% reduction in the time taken to generate a report. This is a significant reduction in the time taken to generate a report and indicates that the developed automated solution is more efficient than the current manual reporting process.

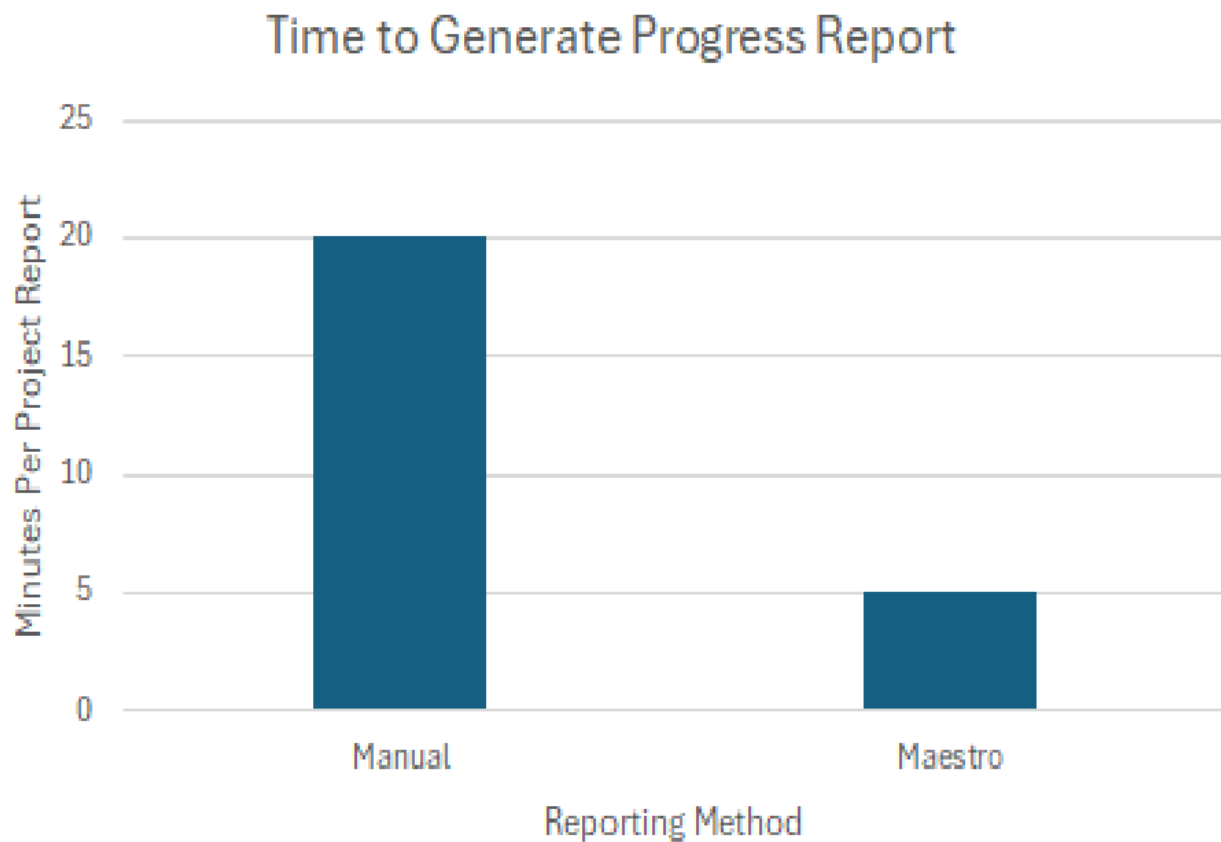


FIGURE 5.4: Time taken to generate a report

To be considered as a viable option to replace the manual reporting process, the automated solution must also be more cost-effective. The only cost related to the manual reporting process is the loss in revenue from the loss of engineering time as a project manager spends time generating reports. The loss in revenue per hour was estimated by Jaegersoft as \$70. The regular ongoing cost of the automated solution was calculated as the cost to deploy the application and the cost of the project manager's time to manage the created report

for a project. Figure 5.5 shows the comparison of the cost per month of the manual and automated reporting processes assuming 5 projects running over the month. The results show that the manual reporting process would cost \$505 while the automated process would cost \$141. This is a 72% reduction in cost associated with reporting and indicates that the developed automated solution is considerably more cost-effective than the manual reporting process.

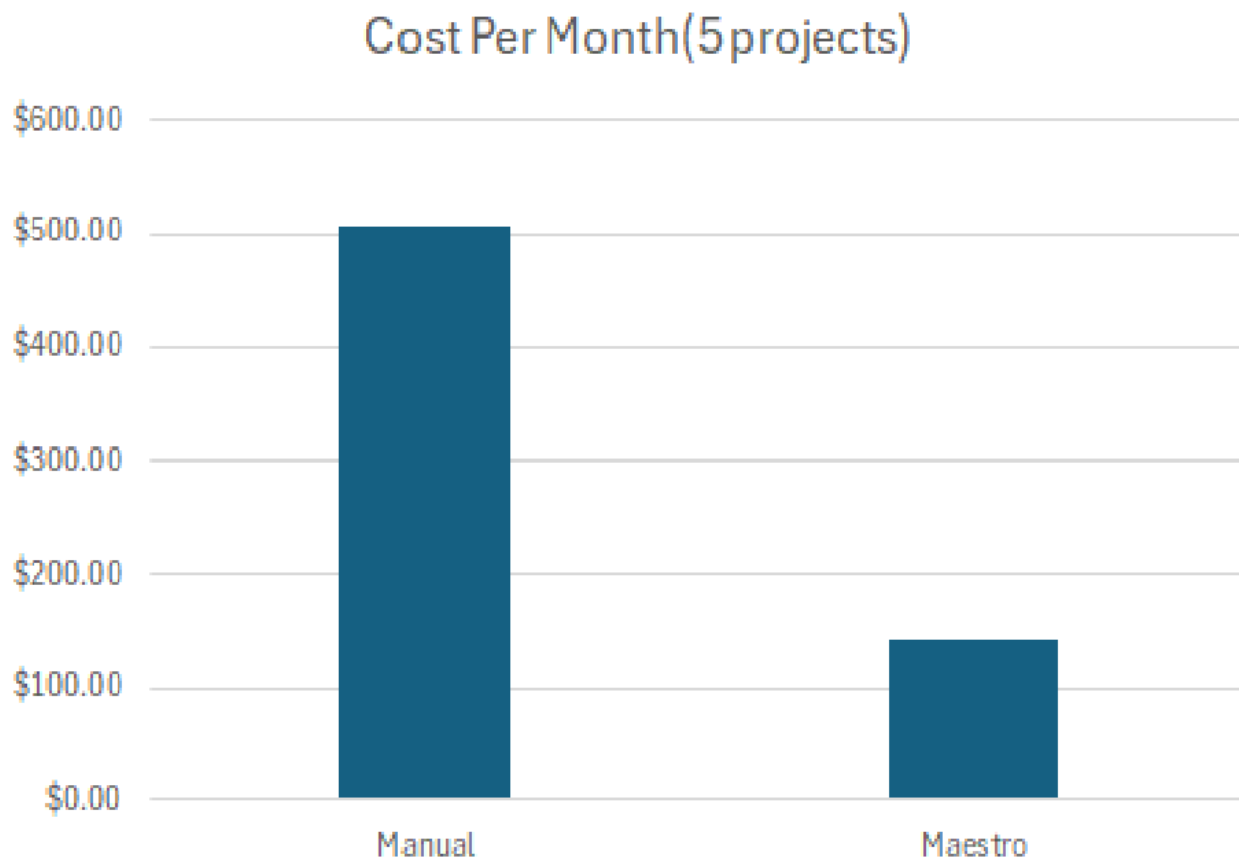


FIGURE 5.5: Cost per month with 5 projects

5.3 Summary

The results of the usability testing session and the concurrent usage assessment were presented and discussed. The results of the usability testing session showed that the application was well received by the testing participants and was considered to be easy to use, well-integrated and easy to learn. There were, however, slight problems with facets of the interface that could be improved to increase the usability of the application. The results of the concurrent usage assessment showed that the application was considerably more efficient and cost-effective than the current manual reporting process. The application was found to be 75% more efficient with time usage than the manual reporting process and 72% more cost-effective. The results of the usability

testing and the concurrent usage assessment suggest that an automated solution for report generation using Clockify data is a viable solution and could be a valuable addition to the current reporting process at Jaegersoft. The next section will discuss the implications of the research and provide recommendations for future research in the area.

6

Conclusions

The conclusions chapter will summarise the findings of the research and the evaluation of the success of the developed application. The chapter will also discuss the implications of the research and provide recommendations for future research in the area.

6.1 Summary

This project set out to investigate the feasibility of an automated solution using Clockify data for progress reporting in a small software consultancy. The project aimed to develop a proof of concept application that could generate weekly progress reports for clients using data from Clockify. The project then used this application to evaluate the effectiveness of the developed solution and the feasibility of the concept as a whole. The project was successful in developing the proof of concept application and evaluating the effectiveness of the developed solution. The objectives of the research were to answer the following questions:

Objective 1: Is an automated solution for report generation using Clockify data suitable for status reports?

Objective 2: Is an automated solution more efficient for the business?

Objective 3: Is an automated solution more cost-effective for the business?

Objective 4: Can an automated solution meet the needs of a client in the context of status reporting?

The project found that it is likely that an automated solution for report generation using Clockify data is suitable for status reporting and can meet the needs of the client. The application was well received by the testing participants and was found to be easy to use, well-integrated and easy to learn. The testing found that the interface was slightly confusing in some areas and that the navigation could be improved. However, the application as a whole was considered to be a positive replacement for the current manual reporting process. The project also found that the developed solution was considerably more efficient and cost-effective than the current manual reporting process. The automated solution was found to be 75% more efficient with time usage than the manual reporting process and 72% more cost-effective. Overall, the project found that an automated solution for report generation using Clockify data is a viable solution and could be a valuable addition to the current reporting process at Jaegersoft.

6.2 Contributions

The results of this research provide a valuable contribution to the software engineering field. The research has demonstrated that an automated solution for report generation using Clockify data is a viable and effective approach. This is a significant contribution as the generation of reports is a common task that is often time-consuming and prone to errors.

In addition to the research conducted on the developed application, the collaboration with Jaegersoft to develop this solution has provided a practical case study for other small software consultancies to follow. The detailed documentation of the development process, challenges faced, and solutions implemented serves as a valuable resource for other developers working on similar projects.

Furthermore, the project has highlighted the potential for significant time and cost savings through the automation of other repetitive tasks. This can be a major benefit for small software consultancies and other businesses, allowing them to allocate more resources to core development activities and improve overall efficiency.

6.3 Implications

The implications of this research are both practical and theoretical. From a practical perspective, the research demonstrates that automating the generation of status reports using data from Clockify can significantly reduce the time and cost associated with manual reporting processes. This has direct implications for small software consultancies like Jaegersoft, as it allows them to allocate more resources to development activities rather than administrative tasks. The implementation of such an automated solution can lead to increased efficiency, reduced operational costs, and improved client satisfaction due to more timely and accurate reporting.

Theoretically, this research contributes to the body of knowledge on the use of automation in project management and reporting. It provides evidence that automated solutions can effectively replace manual processes in the context of status reporting, which can be extended to other areas of project management. The research also highlights the importance of user interface design and usability in the success of automated solutions, suggesting that future research should focus on improving these aspects to enhance the overall effectiveness of such systems.

Overall, the implications of this research are significant for both practitioners and researchers. For practitioners, the findings provide a practical framework for implementing automated solutions to improve efficiency and reduce costs. For researchers, the study offers a foundation for further exploration and development of automated solutions in project management and reporting, emphasising the critical role of user interface design and usability in these systems.

6.4 Limitations

Although the research successfully achieved its objectives, several limitations should be noted. Firstly, the sample size for both the usability testing and the concurrent usage assessment was relatively small. The usability testing involved only 5 participants, while the concurrent usage assessment included just 2 participants. While this sample size was adequate for the scope of this study, a larger sample size would enhance the robustness and generalisability of the findings.

Additionally, the study was confined to employees of Jaegersoft. Although this was suitable for evaluating the project manager's interface and obtaining feedback on behalf of client users, incorporating external clients in future research would be advantageous. This would provide a more comprehensive evaluation of the application's effectiveness and usability from the client's perspective.

Another limitation was the brief duration of the concurrent usage assessment, which spanned only 2 weeks with 2 Jaegersoft employees. While this period was sufficient to estimate the time and cost savings of the automated solution, a longer study period would yield more extensive data on the application's performance and efficiency over time. Including external clients in future studies would also offer a more accurate assessment of the application's ongoing costs and benefits.

6.5 Recommendations

Future research should focus on addressing the limitations of this study to enhance the generalisability and applicability of the findings. This includes expanding the sample size for usability testing and concurrent usage assessment, involving external clients in the evaluation process, and extending the study period to gather more comprehensive data on the application's performance and efficiency.

It would also be valuable to conduct a follow-up study to assess the long-term impact of the automated solution on Jaegersoft's reporting processes. This would involve monitoring the application's performance over an extended period, collecting feedback from clients and employees, and evaluating the ongoing costs and benefits of the automated solution. This longitudinal study would provide valuable insights into the sustainability and effectiveness of the application in the long run.

This research highlights the potential for automating frequently performed tasks in small businesses. Future studies could investigate the creation of additional automated solutions for other repetitive tasks in project management and reporting. This could include automating resource allocation, capacity management, or risk assessment, among other areas. By developing a comprehensive suite of automated tools, small businesses could greatly enhance their operational efficiency and reduce costs.

References

- [1] J. Rakos, K. Dhanraj, S. Kennedy, L. Fleck, S. Jackson, and J. Harris. *The Practical Guide to Project Management Documentation*. John Wiley & Sons, Incorporated, New York, 2004. Available from: ProQuest Ebook Central. [3 June 2024]. [8](#)
- [2] W. Dow and B. Taylor. *Project Management Communications Bible*. John Wiley & Sons, Incorporated, Hoboken, 2008. Available from: ProQuest Ebook Central. [3 June 2024]. [8](#)
- [3] A.P. Snow and M. Keil. The challenge of accurate software project status reporting: A two-stage model incorporating status errors and reporting bias. *IEEE*, 2002. [8](#)
- [4] A.P. Snow, M. Keil, and L. Wallace. The effects of optimistic and pessimistic biasing on software project status reporting. *Information & Management*, 44(2):130–141, 2007. [9](#)
- [5] E. Kupiainen, M.V. Mäntylä, and J. Itkonen. Using metrics in agile and lean software development – a systematic literature review of industrial studies. *Information and Software Technology*, 62:143–163, 2015. [9](#)
- [6] K.V. Padmini, H.M. Dilum Bandara, and I. Perera. Use of software metrics in agile software development process. In *2015 Moratuwa Engineering Research Conference (MERCon)*, 2015. [9](#)
- [7] M.P. Boerman et al. Measuring and monitoring agile development status. In *2015 IEEE/ACM 6th International Workshop on Emerging Trends in Software Metrics*, 2015. [9](#)
- [8] Ahmad Jrade and Julien Lessard. An integrated bim system to track the time and cost of construction projects: A case study. *Journal of Construction Engineering*, 2015(1):579486, 2015. [10](#)
- [9] Brandon Scott-Hill, Craig Anslow, Jennifer Ferreira, Martin Kropp, Magdalena Mateescu, and Andreas Meier. Visualizing progress tracking for software teams on large collaborative touch displays. In *2020 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pages 1–5, 2020. [10](#)